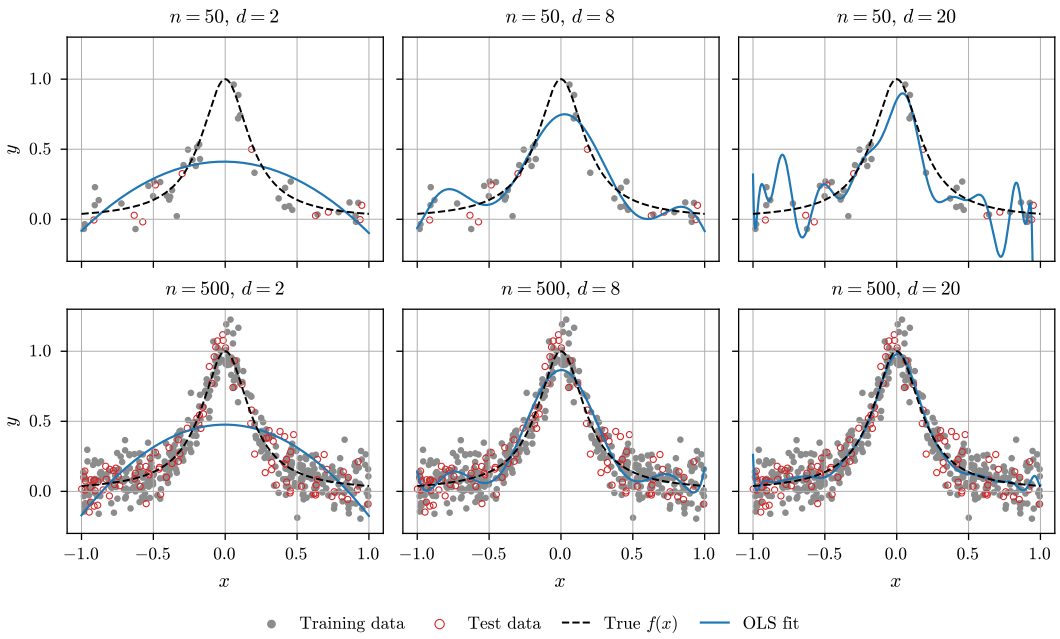
All You Need is Lasso (Or Maybe Not): A Numerical Study of Linear Regression, Resampling and Gradient Descent

Adrian Gjersvik Gundersen,* Victor Berge Johansen,† Casper André Johnsen,‡ and Erica Løken§
Department of Physics and Department of Mathematics, University of Oslo

(Dated: October 7, 2026)

Linear regression is a building block of modern machine learning and, despite its simplicity, captures many of the field's central challenges, such as overfitting, regularization, and optimization. We study polynomial regression applied to Runge's function with Gaussian noise, using Ordinary Least Squares (OLS) and Ridge regression via singular value decomposition, and Lasso regression using iterative optimization. Bootstrap resampling and k-fold cross-validation are used to study the bias–variance tradeoff and to select the polynomial degree and regularization parameter λ. For OLS, both methods give polynomial degree at a neighborhood around $d = 15$ to $d = 18$, with a mean squared error (MSE) of order $\sim 10^{-2}$, close to the noise floor $\sigma^2 = 0.01$. Cross-validation finds no benefits for Ridge and the best Ridge model was essentially OLS, but proved more robust against overfitting. We also implement gradient-descent optimization of the cost functions using both analytical gradients and automatic differentiation, finding that they yield identical results and converge for a learning rate below the theoretical limit, $\gamma < 2/\mu_{\max}(\boldsymbol{H})$. We also extend with adaptive methods and find that momentum converges the fastest. AdaGrad is the most robust, while RMSProp stalls near 10^{-4}. Stochastic gradient descent gave worse results and stalled at 10^{-2}–10^{-1} relative error. Neither did it provide a speedup for our small data set. Finally, we compare all methods and find that Lasso performs slightly worse than OLS and Ridge for our tested values, but further research is needed to conclude. For further research one could study larger grids and more iterations for Lasso.



The above figure provides a visual motivation for the objectives investigated in this report. It shows the true Runge function (dashed curve) together with polynomial OLS fits for two substantially different sample sizes. As the polynomial degree d increases, the model transitions from underfitting to increasingly overfitted behavior, while increasing the number of data points has a stabilizing effect on the fit. These features motivate the investigation of model complexity, the bias–variance tradeoff, and regularized regression methods.

* adriangg@uio.no
† victorbj@uio.no
‡ casperaj@uio.no
§ erical@uio.no

I Introduction **3**

II Theory **3**
A Linear regression model . . . 3
B Model evaluation . . . 3
C Regression methods . . . 4
1 Ordinary least squares . . . 4
2 Ridge regression . . . 4
3 Lasso regression . . . 5
D Data splitting and standardization . . . 5
1 Train–test split . . . 5
2 Standardization . . . 5
E Bias–variance decomposition . . . 5
F Resampling methods . . . 6
1 Bootstrap . . . 6
2 Cross-validation . . . 6
G Gradient-based optimization . . . 6
1 Gradient descent . . . 6
2 Automatic differentiation . . . 7
3 Momentum . . . 7
4 Adaptive learning rate methods . . . 7
5 Stochastic gradient descent . . . 8

III Methods **8**
A Data generation . . . 8
B Data preprocessing . . . 8
C Resampling methods implementation . . . 8
1 Bootstrap . . . 8
2 Cross-validation . . . 9
D Regression implementation . . . 9
E Numerical optimization algorithms . . . 9
F Tools . . . 10

IV Results and discussion **10**
A Investigating model complexity with OLS 10
B Effect of Ridge regularization . . . 11
C Bias–variance tradeoff using bootstrap for OLS . . . 12
D Cross–validation for selecting model . . . 14
Optimal model complexity using OLS . . . 14
Optimal model complexity using Ridge . . . 15
E Gradient descent for OLS and Ridge . . . 16
F Momentum and adaptive optimization methods . . . 17
G Lasso regression . . . 19
H Stochastic gradient descent . . . 20
I Comparison of OLS, Ridge and Lasso . . . 21

V Conclusion **24**

Acknowledgments **25**

References **26**

A Derivation of bias–variance decomposition **26**

B Proof of convergence of gradient descent **29**

C Gradient descent results **34**

D Stochastic gradient decent results **35**

E Use of AI/LLM Tools **36**

Appendix: Use of AI/LLM Tools **36**
1 Tools Used . . . 36
2 Text Contributions . . . 36
3 Code Contributions . . . 36

I. INTRODUCTION

Machine learning provides methods for extracting patterns from some data and using them to make predictions about unseen observations. The field of machine learning has grown broad and complex since its beginnings in the mid-twentieth century [1]. A fundamental machine learning method is linear regression, which involves constructing a model that is linear in its parameters and approximates an underlying function from observed data. A central challenge is to construct a model that is sufficiently complex to capture the underlying function without overfitting to random noise in the training data. This balance is described by the well known bias–variance tradeoff.

In this work, polynomial regression is investigated using training data generated from Runge's function with added independent Gaussian noise. The regression methods employed are Ordinary Least Squares (OLS), Ridge, and Lasso regression to construct predictive models and investigate the effects of complexity and regularization. A model's performance is evaluated using the mean squared error (MSE) along with the coefficient of determination, R^2. The bias–variance tradeoff is studied using bootstrap resampling, while k-fold cross-validation is used to investigate model complexity and select the optimal polynomial degree and regularization parameter λ. The closed-form OLS and Ridge solutions are implemented numerically using singular value decomposition, while Lasso regression is implemented using gradient-based optimization and compared against Scikit-Learn's coordinate-descent implementation. The regression methods are compared using cross-validation to assess how model complexity, regularization, resampling, and optimization influence predictive performance.

Another objective is to investigate iterative optimization methods as an alternative to closed-form regression solutions. Gradient descent (GD) is studied using both analytical gradients and automatic differentiation, and their numerical results are compared with the corresponding closed-form solutions for OLS and Ridge. The dependence of the learning rate on the convergence of GD is studied and compared with the theoretical limit. Optimization is extended to momentum and adaptive methods AdaGrad, RMSProp, and Adam, which are compared in terms of convergence. Finally, we investigate stochastic gradient descent (SGD) using mini-batches and learning rate schedules.

Section II presents the theoretical background for linear regression, OLS, Ridge and Lasso regression, bias–variance decomposition, and gradient-based optimization. Section III describes the numerical implementation of regression methods, data generation, preprocessing, and resampling. The results are presented and discussed in Section IV. Finally, Section V summarizes the main findings and discusses possible extensions. The code developed is available via the UiO GitHub repository [1].

II. THEORY

A. Linear regression model

We use Runge's function,

$$f(x) = \frac{1}{1+25x^2}, \quad x \in [-1,1], \tag{1}$$

as an example problem for investigating different linear regression methods. This function was introduced by Carl Runge to demonstrate that high-degree polynomial interpolation at equally spaced points can induce large oscillations near the interval boundaries [2]. This is known as Runge's phenomenon and reveals that higher-degree polynomials does not necessarily improve accuracy.

The observed targets are generated according to

$$\boldsymbol{y} = \boldsymbol{f} + \boldsymbol{\varepsilon}, \qquad \varepsilon_i \sim \mathcal{N}(0,\sigma^2), \tag{2}$$

where $\boldsymbol{f} = f(\boldsymbol{x})$ is Runge's function evaluated at inputs $\boldsymbol{x}$, and $\boldsymbol{\varepsilon}$ is independent Gaussian noise with zero mean and variance σ^2. A polynomial model of degree d is linear in its $p = d+1$ coefficients and predicts

$$\tilde{y}_i = \sum_{j=0}^{d} \theta_j x_i^j,$$

where θ_j are the regression parameters and $\tilde{y}_i$ the model prediction at x_i. In matrix form,

$$\tilde{\boldsymbol{y}} = \boldsymbol{X}\boldsymbol{\theta}, \tag{3}$$

where $\tilde{\boldsymbol{y}} \in \mathbb{R}^n$, $\boldsymbol{\theta} \in \mathbb{R}^p$ and $\boldsymbol{X} \in \mathbb{R}^{n\times p}$ is the design matrix,

$$\boldsymbol{X} = \begin{pmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^d \\ 1 & x_2 & x_2^2 & \cdots & x_2^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^d \end{pmatrix},$$

with rows corresponding to observations and columns to features.

B. Model evaluation

The model performance is evaluated using the mean squared error,

$$\mathrm{MSE}(\boldsymbol{y},\tilde{\boldsymbol{y}}) = \frac{1}{n}\sum_{i=0}^{n-1}(y_i - \tilde{y}_i)^2,$$

[1] https://github.uio.no/victorbj/FYS-STK4155

where n is the number of evaluated data points, y_i is the observed target, and $\tilde{y}_i$ is the corresponding prediction.

The coefficient of determination measures the performance relative to predicting the mean target value, and is defined as

$$R^2(\boldsymbol{y}, \tilde{\boldsymbol{y}}) = 1 - \frac{\sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2}{\sum_{i=0}^{n-1} (y_i - \bar{y})^2}, \quad \bar{y} = \frac{1}{n} \sum_{i=0}^{n-1} y_i,$$

where $\bar{y}$ is the mean of the observed targets. An R^2 score equal to unity indicates perfect predictions, while $R^2 = 0$ reveals that the model does not perform better than always predicting the mean. A negative score indicates that the model performs worse than a constant model that always predicts the mean.

C. Regression methods

Linear regression methods are used to find the coefficients, $\boldsymbol{\theta}$, of the linear model in Eq. (3). This study employs the linear regression methods OLS, Ridge, and Lasso. These methods estimate the coefficients using different approaches, but are all based on minimizing a cost function, $C(\boldsymbol{X}, \boldsymbol{\theta})$, which we define as $C(\boldsymbol{\theta})$. The cost function measures the gap between the observed data, $\boldsymbol{y} \in \mathbb{R}^n$, and the model predictions, $\tilde{\boldsymbol{y}} = \boldsymbol{X}\boldsymbol{\theta}$, with optimal parameters

$$\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta} \in \mathbb{R}^p} C(\boldsymbol{\theta}).$$

1. Ordinary least squares

OLS minimizes the mean squared error between the observed targets and the model predictions,

$$C_{\text{OLS}}(\boldsymbol{\theta}) = \frac{1}{n} \left\| \boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta} \right\|_2^2,$$

where $\| \cdot \|_2$ denotes the ℓ^2 norm. Differentiating with respect to the parameters $\boldsymbol{\theta}$ yields

$$\nabla_{\boldsymbol{\theta}} C_{\text{OLS}} = \frac{2}{n} \boldsymbol{X}^T (\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y}). \tag{4}$$

Setting the gradient to zero gives the normal equations,

$$\boldsymbol{X}^T \boldsymbol{X} \boldsymbol{\theta} = \boldsymbol{X}^T \boldsymbol{y}.$$

If $\boldsymbol{X}^T \boldsymbol{X}$ is invertible, then the solution is

$$\hat{\boldsymbol{\theta}}_{\text{OLS}} = (\boldsymbol{X}^T \boldsymbol{X})^{-1} \boldsymbol{X}^T \boldsymbol{y}, \tag{5}$$

The Hessian $\frac{2}{n} \boldsymbol{X}^T \boldsymbol{X}$ is positive semidefinite, so the cost function is convex and every stationary point is a global minimum, as shown in the proof of Corollary 3 in Appendix B.

The condition number of the Hessian is its largest eigenvalue divided by its smallest,

$$\kappa(\boldsymbol{H}) = \frac{\mu_{\max}}{\mu_{\min}} = \kappa(\boldsymbol{X})^2,$$

assuming that $\boldsymbol{X}$ has full column rank [3, Eq. 4.27]. A large condition number means that the matrix is ill-conditioned. An ill-conditioned design matrix produces an even more ill-conditioned Hessian, causing the optimal parameters to be sensitive to small perturbations in the data.

2. Ridge regression

Ridge is a regularized version of OLS in which a squared ℓ^2 penalty is added to the OLS cost function,

$$C_{\text{Ridge}}(\boldsymbol{\theta}) = \frac{1}{n} \left\| \boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta} \right\|_2^2 + \frac{\lambda}{n} \|\boldsymbol{\theta}\|_2^2, \tag{6}$$

where $\lambda \geq 0$ is the regularization parameter.[2] Differentiating with respect to $\boldsymbol{\theta}$ gives

$$\nabla_{\boldsymbol{\theta}} C_{\text{Ridge}} = \frac{2}{n} \boldsymbol{X}^T (\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y}) + \frac{2\lambda}{n} \boldsymbol{\theta}. \tag{7}$$

Setting the gradient to zero gives the normal equations,

$$\left(\boldsymbol{X}^T \boldsymbol{X} + \lambda \mathbb{I} \right) \boldsymbol{\theta} = \boldsymbol{X}^T \boldsymbol{y},$$

with solution,

$$\hat{\boldsymbol{\theta}}_{\text{Ridge}} = (\boldsymbol{X}^T \boldsymbol{X} + \lambda \mathbb{I})^{-1} \boldsymbol{X}^T \boldsymbol{y}, \tag{8}$$

where $\mathbb{I}$ is the $p \times p$ identity. For $\lambda > 0$, the cost function is strictly convex, so the minimizer is unique, as shown in the proof of Corollary 3 in Appendix B.

The optimal parameters' dependence on λ is evident when substituting the singular value decomposition (SVD) of the design matrix, $\boldsymbol{X} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^T$, in Eq. (8)

$$\begin{aligned} \hat{\boldsymbol{\theta}}_{\text{Ridge}} &= \boldsymbol{V} (\boldsymbol{\Sigma}^T \boldsymbol{\Sigma} + \lambda \mathbb{I})^{-1} \boldsymbol{\Sigma}^T \boldsymbol{U}^T \boldsymbol{y} \\ &= \sum_{i=1}^{r} \frac{\sigma_i}{\sigma_i^2 + \lambda} (\boldsymbol{u}_i^T \boldsymbol{y}) \boldsymbol{v}_i, \end{aligned}$$

where we have used $\boldsymbol{V}\boldsymbol{V}^T = \boldsymbol{V}^T\boldsymbol{V} = \mathbb{I}$, $r = \text{rank}(\boldsymbol{X})$, and σ_i are the nonzero singular values of $\boldsymbol{X}$. Along each singular direction $\boldsymbol{v}_i$, the OLS solution has coefficient $1/\sigma_i$, whereas Ridge has $\sigma_i/(\sigma_i^2 + \lambda) = s_i(\lambda)/\sigma_i$. Thus, Ridge shrinks the OLS coefficients by the factor

$$s_i(\lambda) = \frac{\sigma_i^2}{\sigma_i^2 + \lambda},$$

[2] We scale the penalty by $\frac{1}{n}$, so we obtain the normal equation without this factor. This is the scaling of Eq. (3.44) in the lecture notes [3]. All λ-values in this report will use this scaling.

[3, Sec. 3.8]. Directions in feature space with $\sigma_i^2 \gg \lambda$ are mostly untouched ($s_i \approx 1$), whereas those with $\sigma_i^2 \ll \lambda$ are strongly suppressed ($s_i \approx 0$).

Increasing λ increases the penalty for large coefficients and drives them towards zero. This coefficient shrinkage reduces the susceptibility of the model to random fluctuations and thus reduces the variance of the model at the cost of additional bias. That is, the regularization parameter controls a bias-variance tradeoff.

Moreover, the regularization parameter λ in Ridge improves the condition number of the normal-equation matrix,

$$\kappa_2(\boldsymbol{X}^T\boldsymbol{X} + \mu\mathbb{I}) = \frac{\mu_{\max}(\boldsymbol{X}^T\boldsymbol{X}) + \lambda}{\mu_{\min}(\boldsymbol{X}^T\boldsymbol{X}) + \lambda}.$$

The added λ reduces the spread in its max and min eigenvalues, resulting in improved numerical stability of the optimal parameters.

3. *Lasso regression*

Lasso replaces the squared ℓ^2 penalty in Ridge regression, Eq. (6), with an ℓ^1 penalty,

$$C(\boldsymbol{\theta}) = \frac{1}{n}\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\|_2^2 + \frac{\lambda}{n}\|\boldsymbol{\theta}\|_1. \tag{9}$$

This cost function is convex because both terms are convex, and $\lambda > 0$ gives coercivity, such that a global minimizer exists. However, uniqueness is not generally guaranteed, but when $\boldsymbol{X}$ has full column rank the cost is strictly convex.

Moreover, Lasso generally does not have a closed-form solution and the absolute value $|\theta_j|$ is not differentiable at zero. The gradient is therefore described using the subdifferential [3, Sec. 3.9, Eq. 3.65],

$$\partial|\theta_j| = \begin{cases} \{-1\}, & \theta_j < 0, \\ [-1, 1], & \theta_j = 0, \\ \{1\}, & \theta_j > 0. \end{cases}$$

Defining $\boldsymbol{r} = \boldsymbol{y} - \boldsymbol{X}\hat{\boldsymbol{\theta}}$, the Lasso optimality conditions are

$$2\,\boldsymbol{x}_j^T\boldsymbol{r} \begin{cases} = \lambda\,\mathrm{sgn}(\hat{\theta}_j), & \hat{\theta}_j \neq 0, \\ \in [-\lambda, \lambda], & \hat{\theta}_j = 0, \end{cases}$$

where $\boldsymbol{x}_j$ is the jth column of $\boldsymbol{X}$ [3, Sec. 3.9, Proposition 3.7]. The interval in the subdifferential at zero allows Lasso to perform feature selection, as the optimality condition still holds for all coefficients equal to zero. For a general design matrix, the coefficients are computed using iterative numerical methods, described in Section II G.

D. Data splitting and standardization

1. *Train–test split*

It is normal to split the data into training and test sets. The training set is used to estimate the model parameters, while the predictive performance of the fitted model is evaluated on the unseen test data. This is done to avoid optimistic estimates of the model performance, as the model may fit the training data well.

2. *Standardization*

In order to ensure fair regularization and improve optimization, the features of the design matrix are standardized by centering and scaling to unit standard deviation. This is particularly important for regularized regression methods such as Ridge and Lasso, while OLS is equivariant under standardization [3, Sec. 3.13]. For features of different scales, the corresponding coefficients may also differ notably in magnitude, resulting in a heavier penalty for some features simply due to their scale. Moreover, standardization may improve the conditioning of the optimization problem, which may accelerate convergence of gradient-based methods as the convergence is slowed by ill-conditioning [3, Sec. 15.3]. To prevent data leakage, the scaling parameters must be estimated from the training set only.

E. Bias–variance decomposition

Consider a model fitted to a training set $\mathcal{L} = \{(x_i, y_i)\}_{i=1}^n$, where y_i is generated according to the model in Eq. (2). Then the expected prediction error of a regression model can be decomposed as

$$\mathbb{E}\left[(\boldsymbol{y} - \tilde{\boldsymbol{y}})^2\right] = \mathrm{Bias}^2[\tilde{\boldsymbol{y}}] + \mathrm{Var}[\tilde{\boldsymbol{y}}] + \sigma^2, \tag{10}$$

where

$$\mathrm{Bias}^2[\tilde{\boldsymbol{y}}] = \mathbb{E}\left[(\boldsymbol{f} - \mathbb{E}[\tilde{\boldsymbol{y}}])^2\right] = \frac{1}{n}\sum_{i=1}^{n}\left(f(x_i) - \mathbb{E}[\tilde{\boldsymbol{y}}]\right)^2, \tag{11}$$

and

$$\mathrm{Var}[\tilde{\boldsymbol{y}}] = \mathbb{E}\left[(\tilde{\boldsymbol{y}} - \mathbb{E}[\tilde{\boldsymbol{y}}])^2\right] = \frac{1}{n}\sum_{i=0}^{n-1}\mathbb{E}\left[(\tilde{y}_i - \mathbb{E}[\tilde{y}_i])^2\right]. \tag{12}$$

The derivation of Eqs. (10), (11), and (12) are given in Appendix A. The expectations defining bias and variance are taken over training data sets, such that $\mathbb{E}[\tilde{\boldsymbol{y}}]$ denotes the average prediction at some input when the model is fitted to different training sets.

Bias measures the deviation between the model prediction averaged over training datasets, $\mathbb{E}[\tilde{\boldsymbol{y}}]$, and the underlying function $\boldsymbol{f}$, evaluated at the same inputs. A model

of insufficient complexity may exhibit high bias as it is unable to capture the behavior of the underlying function. This is associated with underfitting and performs poorly on both the training and test data, regardless of the amount of training data.

The variance measures how strongly the model's predictions change across training sets. High variance indicates sensitivity to the particular training set, such that the model predictions differ notably when the training set is redrawn. This is associated with overfitting. Low variance indicates that predictions remain stable when training sets are redrawn. The final term, σ^2, represents the irreducible error arising from the noise in the observed targets, $\boldsymbol{y}$. This can not be predicted by any model, however well fitted, as the noise is random.

In this study, the underlying function is known. However, in general, the underlying function is unknown and the observed targets $\boldsymbol{y}$ may be used to estimate the squared bias,

$$\mathrm{Bias}^2_{\mathrm{est.}}[\tilde{\boldsymbol{y}}] = \mathrm{Bias}^2_{\mathrm{true}}[\tilde{\boldsymbol{y}}] + \sigma^2. \tag{13}$$

The derivation is given in Appendix A. Thus, replacing the unknown function with the noisy targets causes the noise variance, σ^2, to be absorbed into the estimated squared bias. In this sense, the true model bias does not inherently increase by σ^2, but the estimated bias is polluted by the random noise in $\boldsymbol{y}$.

F. Resampling methods

1. Bootstrap

Bootstrap resampling is used to approximate the expectation values by drawing B training datasets with replacement from the training data. One model is fitted to each of the B resampled training sets and evaluated on the same test set, producing an ensemble of B predictions for each test point. For a test point x_i, the expected prediction is approximated as the average bootstrap prediction,

$$\mathbb{E}[\tilde{y}_i] \approx \frac{1}{B}\sum_{b=1}^{B} \tilde{y}_i^{(b)},$$

where B is the number of bootstrap samples, and $\tilde{y}_i^{(b)}$ is the prediction from bootstrap model b. The variation of the bootstrap predictions estimates the prediction variance. These estimates are used to estimate the squared bias, variance, and mean squared prediction error.

2. Cross-validation

In k-fold cross-validation, the data is shuffled randomly and divided into k folds of approximately equal size. The model is trained k times, using each fold once for testing and the remaining $k-1$ folds for training. The MSE is evaluated on the test fold for each iteration, and the mean MSE across all folds estimates the predictive performance. The standard deviation of the MSE across all folds describes the variance across folds. This reduces the dependence on a particular train–test split. Standardization parameters for the preprocessing must be computed from the training folds only to prevent information leakage into the test set.

G. Gradient-based optimization

Gradient-based methods seek to solve optimization problems through iterative parameter updates, in cases where closed form solutions are impractical or expensive. As shown in Sec. II C 1 and Sec. II C 2, OLS and Ridge require inversion of possibly large matrices which is computationally expensive. Furthermore, Lasso requires iterative optimization, as it does not have a closed-form solution. We employ gradient-based methods to iteratively update the parameters in order to minimize the respective cost function, $C(\boldsymbol{\theta})$.

1. Gradient descent

Gradient descent (GD) is the fundamental iterative method which the others we discuss are built upon. Starting from an initial guess, $\boldsymbol{\theta}_0$, the parameters are updated in the direction of the steepest descent,

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \gamma \nabla_{\boldsymbol{\theta}} C(\boldsymbol{\theta}_k), \tag{14}$$

where $\gamma > 0$ is the learning rate controlling the step size. For OLS and Ridge, the gradients of the cost functions are given in Eqs. (4) and (7), respectively, so we can apply the method directly. For Lasso, we replace the gradient by a subgradient.

The learning rate controls convergence of the method. An excessively large learning rate may overshoot the minimum, whereas an excessively small rate may delay convergence. As shown in Appendix B, if the gradient of the cost function is convex, it's gradient is Lipschitz continuous with constant L, and the function attains its minimum, the gradient descent method is guaranteed to converge for a constant learning rate $\gamma \in (0, 2/L)$. For OLS and Ridge, this is

$$L_{\mathrm{OLS}} = \mu_{\max}(\boldsymbol{H}_{\mathrm{OLS}}), \qquad L_{\mathrm{Ridge}} = L_{\mathrm{OLS}} + \frac{2\lambda}{n},$$

where $\lambda_{\max}(\boldsymbol{H}_{\mathrm{OLS}})$ is the largest eigenvalue of the OLS Hessian matrix. In addition, we show in Appendix B that the method converges with at least $\mathcal{O}(1/k)$ for a convex function.

2. Automatic differentiation

The gradient of the cost function may be impractical to calculate analytically. Automatic differentiation (AD) serves as an alternative to symbolic and finite-difference differentiation, with accuracy up to machine precision. Derivatives are computed by applying the chain rule to the elementary arithmetic operations used to evaluate a function [3, Sec. 4.14]. In contrast to symbolic differentiation, AD evaluates the derivative without constructing a closed-form expression. Moreover, finite-difference methods approximate the derivative using small perturbations, which introduces a truncation error that depends on the step size. AD avoids these issues by propagating exact derivatives through the program using the chain rule.

AD operates in forward or reverse mode. Forward mode computes the derivatives alongside function values, typically at the cost of p sweeps to obtain the full gradient with respect to p parameters [3, Sec. 4.14.3]. Reverse mode first evaluates the function and then propagates the derivatives backwards to obtain the full gradient. This typically costs a small constant multiple of the cost of a single evaluation of the cost function, but has a higher memory usage for storage of intermediate values.

3. Momentum

A problem with the GD method in Eq. (14) is that it can converge quite slowly at rate $\mathcal{O}(1/k)$, as proved in Corollary 1 in Appendix B. A way to fix this is by introducing a velocity term, $\boldsymbol{v}_k$, updated as

$$\boldsymbol{v}_{t+1} = \beta \boldsymbol{v}_t + \nabla_{\boldsymbol{\theta}} C(\boldsymbol{\theta}_t), \qquad \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \gamma \boldsymbol{v}_{t+1},$$

where $\beta \in [0, 1)$ is the momentum parameter, commonly chosen close to 0.9. This is also called the heavy ball method, as it can be seen as analogous to a heavy ball rolling downhill. The gradients pointing in the same direction accumulate and accelerate the ball.

4. Adaptive learning rate methods

Both GD and momentum use a single learning rate for all parameters. Adaptive learning rate methods instead assign each parameter its own effective learning rate, determined by the history of its gradients. A common adaptive method is AdaGrad, which rescales the learning rate based on gradients from previous iterations. The gradient is scaled by the inverse square root of the diagonal Hessian,

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \gamma \boldsymbol{H}^{-1/2} \nabla_{\boldsymbol{\theta}} C_{i_t}(\boldsymbol{\theta}_t),$$

where $\nabla C_{i_t}(\boldsymbol{\theta}_t)$ is denoted as $\boldsymbol{g}_t$ for convenience. Since the Hessian is expensive to compute, AdaGrad instead approximates its diagonal elements by an accumulate relation for $\boldsymbol{r}_t$, which is defined as

$$\boldsymbol{r}_t = \boldsymbol{r}_{t-1} + \boldsymbol{g}_t \odot \boldsymbol{g}_t = \boldsymbol{r}_{t-1} + \boldsymbol{g}_t^2, \tag{15}$$

where $\odot$ is the Hadamard product [4]. The vector $\boldsymbol{r}_t$ contains the accumulated squared gradient for each parameter. This is a cheaper per-parameter estimate of how steep the cost is in that direction. The update for the AdaGrad method becomes

$$\boldsymbol{\theta_{t+1}} = \boldsymbol{\theta_t} - \frac{\gamma}{\sqrt{\boldsymbol{r_t}} + \varepsilon} \boldsymbol{g}_t,$$

where a small constant ε is added to ensure a non zero denominator. Here, ε is outside the root, which is the same convention as PyTorch [5–7]. The issue with AdaGrad is the accumulative relation of $\boldsymbol{r_t}$ in Eq. (15). This makes the effective learning rates non-increasing, which can be fatal as they may become very small before a minimum is reached. That is, the sum has infinite memory and all previous gradients affect the current state.

RMSProp addresses this problem by replacing the cumulative relation of Eq. (15) with an exponentially decaying average,

$$\boldsymbol{v}_t = \rho \boldsymbol{v}_{t-1} + (1 - \rho)(\nabla_{\boldsymbol{\theta}} C(\boldsymbol{\theta}_t))^2, \tag{16}$$

where the parameter $\rho \in [0, 1)$ is chosen. The RMSProp update is

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\gamma}{\sqrt{\boldsymbol{v}_t} + \varepsilon} \nabla_{\boldsymbol{\theta}} C(\boldsymbol{\theta}_t).$$

Unrolling the recursion in Eq. (16) for $\boldsymbol{v}_0 = \boldsymbol{0}$ gives

$$\boldsymbol{v}_t = (1 - \rho) \sum_{i=1}^{t} \rho^{\,t-i} \boldsymbol{g}_i^2. \tag{17}$$

Thus, older gradients are weighted down geometrically, which ensures that $\boldsymbol{v}_t$ does not grow without bound. If the gradients in a direction become smaller, $\boldsymbol{v}_t$ also decreases, allowing the effective learning rate $\gamma/(\sqrt{\boldsymbol{v}_t} + \varepsilon)$ to increase again. This removes the main weakness of AdaGrad, where the effective learning rate is non-increasing.

Since each component of the gradient is divided by the root of its own running average, the step length becomes roughly γ in each parameter, regardless of the gradient's scale. This is particularly helpful when the gradient components differ by orders of magnitude. However, with a constant learning rate, the step size does not shrink near the minimum. Then, the iterates may oscillate around the minimum rather than settling, unless the learning rate is decayed. For the initialization $\boldsymbol{v}_0 = \boldsymbol{0}$, the first iteration of RMSProp may be too large, as its magnitude is $\gamma/\sqrt{1 - \rho}$, which is an issue for values of ρ close to unity.

Adaptive moment estimation (Adam) combines momentum and RMSProp to get fast convergence and proper

scaling. It introduces the first- and second-moment estimates $\boldsymbol{m}_t$ and $\boldsymbol{v}_t$, defined as

$$\boldsymbol{m}_t = \beta_1 \boldsymbol{m}_{t-1} + (1-\beta_1)\nabla_{\boldsymbol{\theta}} C(\boldsymbol{\theta}_t),$$

$$\boldsymbol{v}_t = \beta_2 \boldsymbol{v}_{t-1} + (1-\beta_2)(\nabla_{\boldsymbol{\theta}} C(\boldsymbol{\theta}_t))^2,$$

where $\beta_1, \beta_2 \in [0,1)$, and $\boldsymbol{m}_0, \boldsymbol{v}_0$ are chosen. The second-moment has the same form as Eq. (16), with β_2 taking the place of ρ.

Following the series of the first-moments, one can find the relation $\boldsymbol{m}_t = (1-\beta_1)\sum_{i=1}^{t} \beta_1^{t-i} \boldsymbol{g}_i$. Assuming stationary gradient, then $\mathbb{E}(\boldsymbol{m}_t) = \mathbb{E}(\boldsymbol{g})(1-\beta_1^t)$ by the geometric sum. A similar result can be found for $\boldsymbol{v}_t$ using Eq. (17). As the moments are initialized to zero, they are biased in the first few iterations. To counteract this, they are scaled as

$$\hat{\boldsymbol{v}}_t = \frac{\boldsymbol{v}_t}{1-\beta_2^t}, \qquad \hat{\boldsymbol{m}}_t = \frac{\boldsymbol{m}_t}{1-\beta_1^t}.$$

The update scheme for Adam becomes

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\alpha}{\sqrt{\hat{\boldsymbol{v}}_t} + \varepsilon} \hat{\boldsymbol{m}}_t,$$

where α is the learning rate.

5. *Stochastic gradient descent*

Another caveat of GD is the computational cost of evaluating the gradient across the entire data set. As many cost functions can be written as a sum over data points, $C(\boldsymbol{\theta}) = \sum_i c(x_i, \boldsymbol{\theta})$, evaluating the gradient becomes expensive for large n. Using stochastic gradient descent (SGD), the gradient is estimated using a random subset of the data, B_j, called a mini-batch. For a mini-batch consisting of M data points, the parameters are updated according to

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \gamma_k \, \boldsymbol{g}_{j_k}(\boldsymbol{\theta}_k), \quad \boldsymbol{g}_j(\boldsymbol{\theta}) = \frac{1}{M} \sum_{i \in B_j} \nabla_{\boldsymbol{\theta}} c_i(x_i, \boldsymbol{\theta}).$$

For $M = n$, the update is reduced to ordinary GD. The mini-batch gradient is an unbiased estimator of the full gradient, with noise scaling approximately as $1/\sqrt{M}$ [3, Sec. 4.7]. One pass through all mini-batches is called an epoch, after which the data are reshuffled. An epoch requires approximately the same cost as one full-batch gradient descent step, but performs n/M parameter updates. The stochastic gradient has non-decaying noise, such that the method will remain fluctuating around the minimum rather than converging to it. To reduce these fluctuations, one introduces a decay of the learning rate over time, referred to as a learning rate schedule.

III. METHODS

A. Data generation

The observed target values were generated according to the noisy model in Eq. (2), with $f(x)$ given by Runge's function of Eq. (1). Input values, x_i, were sampled independently from a uniform distribution on the interval $[-1, 1]$,

$$x_i \sim \mathcal{U}(-1, 1), \qquad i = 1, \ldots, n,$$

with $n = 100$ data points unless otherwise stated. The independent Gaussian noise added to each observed target had a default noise level of $\sigma = 0.1$. Input values and noise were generated using `NumPy` [8] with a fixed random seed, `seed` $= 67$, to ensure reproducibility.

B. Data preprocessing

The splitting of the data was performed using `train_test_split` from Scikit-learn [9]. A fixed test size of 0.2 was used, so that 80% of the data was used for training and the remaining 20% for testing. The design matrix and the target values were split using the same shuffled indices to preserve correspondence between inputs and target values. Each feature, except the intercept, was standardized using its training mean and standard deviation,

$$\boldsymbol{x}_j^{\text{scaled}} = \frac{\boldsymbol{x}_j - \bar{\boldsymbol{x}}_j}{\sigma_j},$$

where $\boldsymbol{x}_j$ denotes the jth feature, and $\bar{\boldsymbol{x}}_j$ and σ_j are its training mean and standard deviation, respectively. Each standardized training feature therefore has zero mean and unit standard deviation. The same scaling parameters were used on the test features to prevent leakage. The targets were standardized similarly,

$$\boldsymbol{y}^{\text{scaled}} = \frac{\boldsymbol{y} - \bar{y}_{\text{train}}}{\sigma_{y_{\text{train}}}},$$

where $\bar{\boldsymbol{y}}_{\text{train}}$ and $\sigma_{y_{\text{train}}}$ are the training target mean and standard deviation, respectively. This standardization was applied to both the training and test targets. To compare predictions to the original targets when evaluating performance, the predictions were transformed back to the original scale,

$$\tilde{\boldsymbol{y}} = \tilde{\boldsymbol{y}}^{\text{scaled}} \sigma_{y_{\text{train}}} + \bar{y}_{\text{train}}.$$

C. Resampling methods implementation

1. *Bootstrap*

A dataset of $n = 500$ was divided into training and test sets. From the training set, $B = 100$ bootstrap samples were generated by drawing n_{train} paired observations

(x_i, y_i) with replacement from the training set. This was implemented numerically using `np.random.choice` with `replace=True` [8] to allow repeated indices. A separate regression model was fitted to each bootstrap sample, and evaluated on the same fixed test set. The ensemble of predictions were used to estimate squared bias, variance, and the mean squared prediction error, averaged over test points. The procedure was repeated for polynomial degrees $d = 0, \ldots, 30$.

2. *Cross-validation*

For cross-validation, $k = 5$ and $k = 10$ folds were used, which were generated using `KFold` from Scikit-learn [9]. Before splitting, the data was shuffled randomly using a fixed random seed for reproducibility. A model was fitted to the training folds and tested on the test fold. The mean MSE and its standard deviation was reported across the k folds. A manual implementation of this method was validated against Scikit-learn's `cross_val_score`.

D. Regression implementation

The regression parameters for OLS and Ridge were computed using the SVD of the design matrix using `linalg.svd` from NumPy [8]. This avoids explicit forming and inverting $\boldsymbol{X}^T\boldsymbol{X}$, which can be numerically unstable when $\boldsymbol{X}$ is ill-conditioned. Substituting the SVD of $\boldsymbol{X}$ in Eq. 5 gives

$$\hat{\boldsymbol{\theta}}_{\text{OLS}} = \boldsymbol{V}\,\text{diag}\left(\frac{1}{\sigma_i}\right)\boldsymbol{U}^T\boldsymbol{y},$$

To avoid numerical instability due to division by very small values, singular values below the tolerance

$$\tau_{\text{SVD}} = \max(n, p)\,\epsilon_{\text{mach}}\,\sigma_{\max},$$

were treated as zero, where n is the number of observations, p is the number of features, ϵ_{mach} is the machine precision for double-precision floating-point numbers, and $\sigma_{\max}$ is the largest singular value. This tolerance corresponds to the default rank criterion used by Numpy [8]. When singular values are cut, the solution is then a truncated-SVD solution rather than the exact solution where directions associated with very small singular values are removed.

Substituting the SVD of $\boldsymbol{X}$ into Eq. (8), the Ridge parameters were computed as

$$\hat{\boldsymbol{\theta}}_{\text{Ridge}} = \boldsymbol{V}\,\text{diag}\left(\frac{\sigma_i}{\sigma_i^2 + \lambda}\right)\boldsymbol{U}^T\boldsymbol{y}.$$

Lasso has no closed-form solution, so its parameters were computed iteratively. For $\theta_j = 0$, `jax.grad` returns $+1$ for the derivative of $|\theta_j|$, while our analytic gradient uses $\text{sgn}(0) = 0$. Both values lie in the subdifferential $\partial|\theta_j| = [-1, 1]$ at $\theta_j = 0$ and are therefore valid subgradients, and the two gradients agree for all $\theta_j \neq 0$. Neither choice keeps a coefficient exactly at zero, so subgradient descent does not produce the sparse solutions characteristic of Lasso. In our own Lasso regression implementation, the analytical gradient is used.

Since the Lasso cost function has no closed-form minimizer, we used the coordinate-descent solver in Scikit-Learn, `sklearn.linear_model.Lasso` [10], as a reference for our gradient-based implementation. The two implementations normalize the cost function differently. Scikit-Learn minimizes

$$C_{\text{SK}}(\boldsymbol{\theta}) = \frac{1}{2n}\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\|_2^2 + \alpha\|\boldsymbol{\theta}\|_1,$$

where n denotes the number of training samples used to fit the model. This function equals half of our cost function in Eq. (9) when $\alpha = \frac{\lambda}{2n}$. When comparing the solutions, we took this into account in our scaling. The convergence tolerance was set to 10^{-10} with a maximum of 10^5 iterations.

The intercept was treated consistently in both implementations. Since the features and targets are standardized using training statistics only, the optimal intercept is zero. We therefore fitted Scikit-Learn without the column of ones, using `fit_intercept=False`, and set $\boldsymbol{\theta}_0 = \boldsymbol{0}$.

E. Numerical optimization algorithms

Algorithm 1 describes all optimization methods used in this study through the different parameter choices listed in Table I. Both moment vectors are initialized to zero, $\boldsymbol{m}_0 = \boldsymbol{v}_0 = \boldsymbol{0}$, and $\varepsilon_0 = 10^{-8}$ was used for the adaptive methods. For AdaGrad, $\boldsymbol{v}_t$ represents the accumulated squared gradients, and therefore corresponds to $\boldsymbol{r}_t$ in Eq. (15).

The specific values used for the parameters in Tbl. I are $\beta = \beta_1 = \rho = 0.9$ and $\beta_2 = 0.999$. The learning rates were varied to find the optimal values for each method, and are reported in Sec. IV. The parameters a and b are introduced in the general algorithm to distinguish update rules, with a controlling AdaGrad and b controlling the bias correction used for Adam. For all iterative optimization methods, the initial parameter guess was $\boldsymbol{\theta}_0 = \boldsymbol{0}$.

If the number of data points, n, is not an integer multiple of the mini-batch size, M, the final mini-batch of each epoch contains fewer than M data points. For full-batch gradient descent, $M = n$ and $K = 1$. The mini-batch gradient averages the data term over the $|B_k|$ observations in current the batch, while the penalty term

is scaled by n. This is done so that λ has the same interpretation for all batch sizes. In the full-batch experiments, we set the stopping tolerance to $\tau = 0$ and $t_{\max} = 10^4$, such that every run performs the full 10^4 updates. We however record the amount of iterations it takes to reach 10^{-8} relative error. The learning rate is constant, $s(t) = 1$, except for the stochastic runs using learning rate decay, for which $s(t)$ is defined as

$$s(t) = \frac{t_1}{t + t_1},$$

where t denotes the number of parameter updates, and t_1 is the number of updates after which the learning rate has been halved. This timescale was specified in epochs as E_1, and converted to parameter updates through $t_1 = KE_1$, where $K = \lceil n/M \rceil$ is the number of mini-batch. Each adaptive optimization method was evaluated both with and without scheduling.

Since gradient-based optimization methods can oscillate between updates, we will smooth our plots using a centered moving average of width W. This is only for visual effects and was done using `ndimage.uniform_filter1d` from SciPy [11].

TABLE I: Parameter choices that reduce Algorithm 1 to the respective optimization algorithms.

Method	α	β_1	β_2	ε	a	b	M
Gradient descent	γ	0	1	1	0	0	n
Stochastic GD	γ	0	1	1	0	0	$< n$
Momentum	$\gamma/(1-\beta)$	β	1	1	0	0	$\leq n$
AdaGrad	γ	0	1	ε_0	1	0	$\leq n$
RMSProp	γ	0	ρ	ε_0	0	0	$\leq n$
Adam	α	β_1	β_2	ε_0	0	1	$\leq n$

F. Tools

All numerical experiments were performed on a system equipped with an AMD Ryzen 7 9800X3D CPU, 64 GB of DDR5-6000 MHz RAM, and a NVIDIA GeForce RTX 3080 with 12 GB of VRAM. GPU parallelization was used for the JAX-based computations.

The numerical implementation of all methods was done using Python 3.12. NumPy [8] (version 2.5.2) was employed for array calculations and bootstrap sampling. Scikit-learn [9] (version 1.9.0) was used for data splitting, cross-validation and as a reference Lasso solver, `sklearn.linear_model.Lasso`. Further, SciPy [12] (version 1.18.1) was used for smoothing the error curves. Automatic differentiation was performed using `jax.grad` from JAX [13], which operates in reverse mode (version 0.11.1 for both `jax` and `jaxlib`). Matplotlib [14] (version 3.11.1) was used for the creation of all figures in this report.

Algorithm 1 General adaptive gradient method. Gradient descent, stochastic gradient descent, momentum, AdaGrad, RMSProp, and Adam are recovered by the parameter choices listed in Table I. The choice $\beta_2 = 1$ is included in order to recover methods without an adaptive second-moment. Here, t denotes the number of parameter updates, and one pass through all K mini-batches defines one epoch.

Require: Learning rate α, schedule $s(t)$, constant $\varepsilon > 0$
Require: Decay rates $\beta_1 \in [0, 1)$, $\beta_2 \in [0, 1]$
Require: Accumulation switch $a \in \{0, 1\}$, bias-correction switch $b \in \{0, 1\}$
Require: Batch size $M \leq n$, tolerance $\tau \geq 0$, maximum number of updates $t_{\max}$

$\boldsymbol{\theta}_1 \leftarrow \mathbf{0}$, $\boldsymbol{m}_0 \leftarrow \mathbf{0}$, $\boldsymbol{v}_0 \leftarrow \mathbf{0}$, $t \leftarrow 1$, $K \leftarrow \lceil n/M \rceil$
loop ▷ one epoch
  Shuffle $\{0, \ldots, n-1\}$ and split into K mini-batches
  for $k = 1, \ldots, K$ **do**
    $\boldsymbol{g}_t \leftarrow \nabla C_{B_k}(\boldsymbol{\theta}_t)$ ▷ gradient on the mini-batch
    $\boldsymbol{m}_t \leftarrow \beta_1 \boldsymbol{m}_{t-1} + (1 - \beta_1)\, \boldsymbol{g}_t$ ▷ first moment
    $\boldsymbol{v}_t \leftarrow \beta_2 \boldsymbol{v}_{t-1} + (1 - \beta_2 + a)\, \boldsymbol{g}_t^2$ ▷ second moment
    $\hat{\boldsymbol{m}}_t \leftarrow \boldsymbol{m}_t / \left(1 - b\, \beta_1^t\right)$ ▷ bias correction if $b = 1$
    $\hat{\boldsymbol{v}}_t \leftarrow \boldsymbol{v}_t / \left(1 - b\, \beta_2^t\right)$
    $\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \dfrac{\alpha\, s(t)}{\sqrt{\hat{\boldsymbol{v}}_t} + \varepsilon}\, \hat{\boldsymbol{m}}_t$
    if $\|\boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_t\|_2 < \tau$ **or** $t = t_{\max}$ **then**
      return $\boldsymbol{\theta}_{t+1}$
    $t \leftarrow t + 1$

IV. RESULTS AND DISCUSSION

The main results are presented and discussed in this section. Additional plots and numerical results that support the analysis, but are omitted for brevity, are provided in the supplementary Jupyter notebook, `supplementary_figures.ipynb`, available in the UiO GitHub repository [3]. In addition, when splitting the data, 20% of the data was used as test data. It is worth noting that the MSE and R^2 score are plotted on different axes in the same figure, as their numerical scales differ.

A. Investigating model complexity with OLS

The performance of OLS regression was investigated as a function of polynomial degree, while also exploring the

[3] https://github.uio.no/victorbj/FYS-STK4155

effects of varying the number of data points and the noise level. Fig. 1 shows the test MSE and R^2 score as a function of polynomial degree for $n = 50, 100$ and 500 data points, with fixed noise level $\sigma = 0.1$. In all three cases, the test MSE decreases significantly over the first few degrees. This behavior indicates underfitting at low degrees, in which the model is not sufficiently complex to capture the behavior of Runge's function. For degrees above $d \approx 8$, the MSE exhibits an increasing dependence on the number of data points. The test MSE is fairly stable for $n = 100$ and $n = 500$, while it fluctuates for $n = 50$, with a distinct increase at degree 15. Therefore, more complex models are more sensitive to smaller data sets.

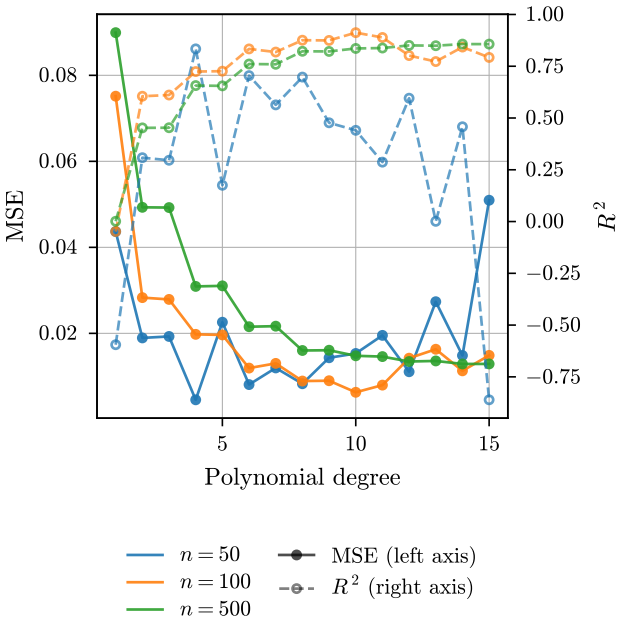


FIG. 1: Test MSE (solid) and R^2 score (dashed) for OLS regression as a function of polynomial degree, for $n = 50$, 100 and 500 data points with noise $\sigma = 0.1$.

The corresponding R^2 scores show the same overall behavior. For $n = 100$ and $n = 500$, the R^2 score increases rapidly at low degrees and stabilizes at roughly 0.8 beyond $d \approx 7$. Contrarily, the score for $n = 50$ varies significantly across polynomial degrees and becomes negative at degree 15. As described in Sec. II B, a negative R^2 score reveals that the model performs worse than predicting the mean of the observed targets. In accordance with the MSE, increasing the number of data points stabilizes the predictions across polynomial degrees as the model complexity increases.

Moreover, the dependence on the noise level is shown in Fig. 2, using $n = 100$ observations. For noiseless data, the MSE decreases monotonically as the polynomial degree increases, with almost perfect predictions for the highest degree considered. Models including noise have an initially decreasing MSE, but it starts to increase after a minimum at $d \approx 10$, after which increasing the complexity provides no improvement. For $\sigma = 0.2$ the MSE is generally higher and shows larger fluctuation, demonstrating the sensitivity of the models to noise. The corresponding R^2 score decreases for higher degrees. That is, increasing the model complexity improves the MSE for lower degrees, but shows no such effect after the degree where the model is sufficiently complex to capture the underlying function. Adding noise to the data makes the model more sensitive to fluctuations for higher polynomial degrees.

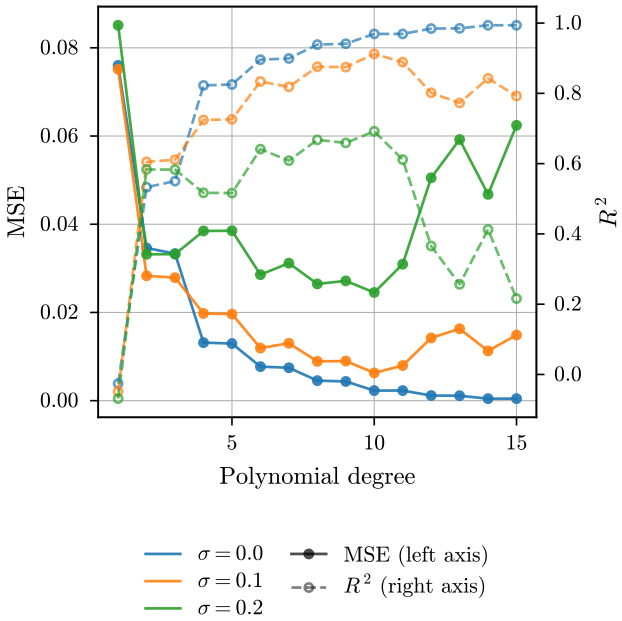


FIG. 2: Test MSE (solid) and R^2 score (dashed) for OLS regression as a function of polynomial degree, for noise levels $\sigma = 0,\ 0.1,\ 0.2$ and $n = 100$ data points.

Altogether, we have observed that the performance of the model is dependent on both the noise level of the data and the number of data points. The predictive performance is improved as the model becomes sufficiently complex, but is still sensitive to the number of data points and noise. As a next step, we investigate regularized regression methods. Moreover, the fitted coefficients as function of the polynomial degree for a fixed n and σ is provided in the Jupyter notebook `supplementary_figures.ipynb`

B. Effect of Ridge regularization

The effect of Ridge regularization was investigated across polynomial degrees for different values of the regularization parameter λ. Figure 3 shows the test MSE and R^2 scores across polynomial degrees for OLS and Ridge with $\lambda = 10^{-4}, 10^{-2}, 1$, and 10^2, using $n = 100$ data points with noise $\sigma = 0.1$. The MSE decreases rapidly over the

first few polynomial degrees for OLS and weakly regularized Ridge models, whereas the R^2 score increases. Beyond $d \approx 6$, the MSE and R^2 scores stabilizes, showing little improvement when further increasing the polynomial degree.

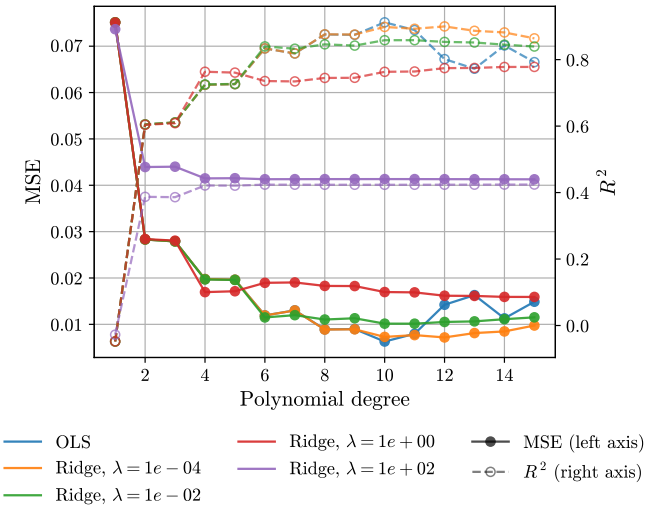


FIG. 3: Test MSE (solid) and R^2 scores (dashed) as a function of polynomial degree for OLS and Ridge regression with different penalty parameters λ. The data set contains $n = 100$ data points with noise level $\sigma = 0.1$.

Ridge follows OLS closely for the least regularized models, with $\lambda = 10^{-4}$ and 10^{-2}, but the Ridge models obtain a lower MSE at higher polynomial degrees. The two strongly regularized models with $\lambda = 1$ and $\lambda = 10^2$ perform significantly worse. For $\lambda = 10^2$, the MSE and the R^2 score become effectively independent of the polynomial degree, such that increasing the complexity does not improve the predictions. This reflects how excessive regularization results in underfitting, as the strict penalty overly restricts the models complexity. Moreover, the fitted coefficients as function of the polynomial degree for a fixed n and σ for OLS and Ridge is provided in `supplementary_figures.ipynb`

The dependence on the penalty parameter is further investigated using a polynomial of degree 15, depicted in Fig. 4. Ridge regression reduces to OLS for very small λ, such that predictive performance does not improve until λ is increased to $\sim 10^{-6}$. Increasing the regularization further reduces the MSE, with the minimum obtained at intermediate values around $\lambda = 10^{-4}$ to 10^{-3}. In accordance, the R^2 score reaches its maximum in this region as well. As λ increases beyond the optimal region, the MSE increases substantially and the R^2 score drops. Hence, regularization has a positive effect for well-chosen values of λ for this polynomial degree, but excessive regularization causes underfitting as the parameters are shrunk toward zero. The optimal λ depends on the problem at hand and may be selected using cross-validation.

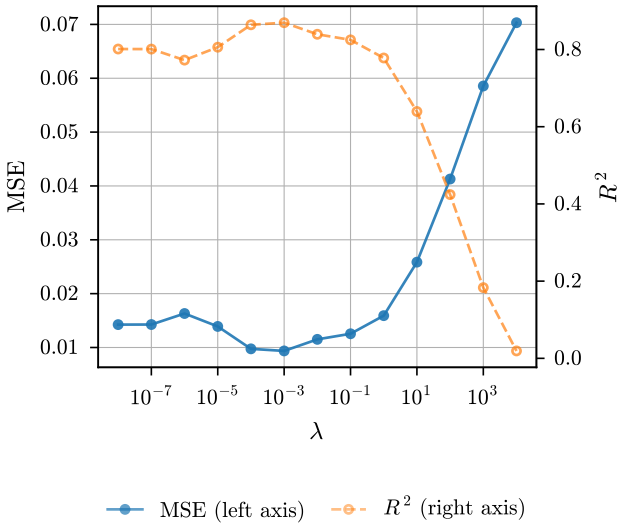


FIG. 4: Test MSE (solid) and R^2 score (dashed) for Ridge regression as a function of the penalty parameter λ, for a polynomial of degree 15. The dataset had $n = 100$ data points with noise $\sigma = 0.1$.

As described in Sec II C 2, the effect of Ridge regularization is evident through the SVD of the Ridge estimator. Fig. 5 shows the shrinkage factor $\sigma_i^2/(\sigma_i^2 + \lambda)$ relative to OLS as a function of the singular modes, σ_i, of a degree 15 design matrix for four penalties. A shrinkage factor of $s_i \approx 1$ means that the singular mode is unchanged from OLS, while $s_i \approx 0$ means that it is strongly suppressed. Ridge suppresses modes with $\sigma_i^2 \ll \lambda$, as shrinkage factor is close to zero. Opposite, for $\sigma_i^2 \gg \lambda$ the modes are retained as the factor approaches unity. For $\sigma_i = \sqrt{\lambda}$, the shrinkage factor is 1/2. As seen in Fig. 5, increasing λ suppresses progressively more modes as the transition is shifted to higher singular values, substantiating the behavior of the MSE for excessively large λ shown in Fig. 4.

C. Bias–variance tradeoff using bootstrap for OLS

To examine underfitting and overfitting, the training and test MSE were compared as functions of polynomial degree for OLS regression. Fig. 6 shows that the training MSE decreases with increasing complexity, whereas the test MSE initially decreases and reaches a minimum at degree 19. At low degrees, the model underfits the data, while at high degrees it is overfitted, as indicated by the high test MSE. Very complex models have a low training error that reaches the noise floor σ^2, while the test error worsens. This indicates that the model is fitting to random noise in the training data, such that highly complex models becomes sensitive to the training dataset.

The bias–variance decomposition of the test MSE was

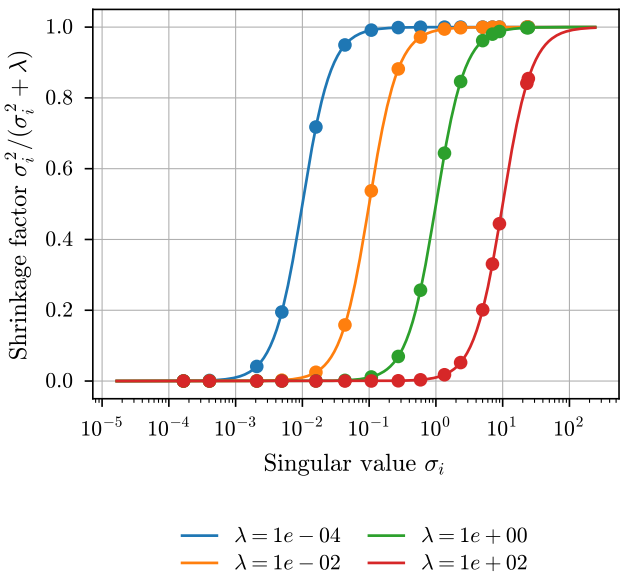


FIG. 5: Ridge shrinkage factor $s_i = \sigma_i^2/(\sigma_i^2 + \lambda)$ as a function of the singular value σ_i for different penalties. The model is a polynomial model of degree 15. Markers indicate the singular values of the design matrix. Modes with $\sigma_i \ll \sqrt{\lambda}$ are strongly suppressed, whereas modes with $\sigma_i \gg \sqrt{\lambda}$ are approximately unchanged. The shrinkage factor equals $1/2$ at $\sigma_i = \sqrt{\lambda}$.

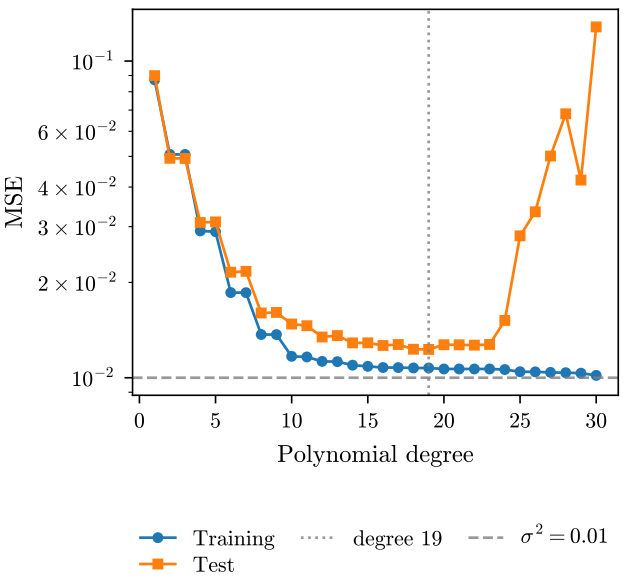


FIG. 6: Training and test MSE for OLS as a function of polynomial degree, with $n = 500$ data points. The dotted line marks the degree with the lowest test MSE, and the dashed line marks the noise variance σ^2 of the data.

studied using both the true Runge function $\boldsymbol{f}$ and the noisy targets $\boldsymbol{y}$ as references, given in Eqs. (1) and (2) respectively. Fig. 7 shows the test error, variance, true squared bias, and measured squared bias as functions of polynomial degree estimated using 100 bootstrap resamples and fixed test set. When using the true function as reference, the squared bias decreases with increasing polynomial degree at the cost of increased variance, illustrating the bias–variance tradeoff. At approximately degree 15, the two are of comparable magnitude, after which the variance dominates. For high degrees, the increasing test error and variance indicate overfitting. For $d \gtrsim 25$, both the true squared bias and the measured squared bias increases rapidly, together with the variance. The increase in bias should not be interpreted as an intrinsic increase in the theoretical bias, but rather as being caused by numerical instability. At very high polynomial degrees, the design matrix becomes severely ill-conditioned, such that small singular values are truncated and the solution is no longer the full OLS solution. Moreover, the bias estimate may also be affected by the mean prediction computed across the bootstrap models, in which numerically unstable high-degree models may distort the mean value, which directly affects the bias estimate.

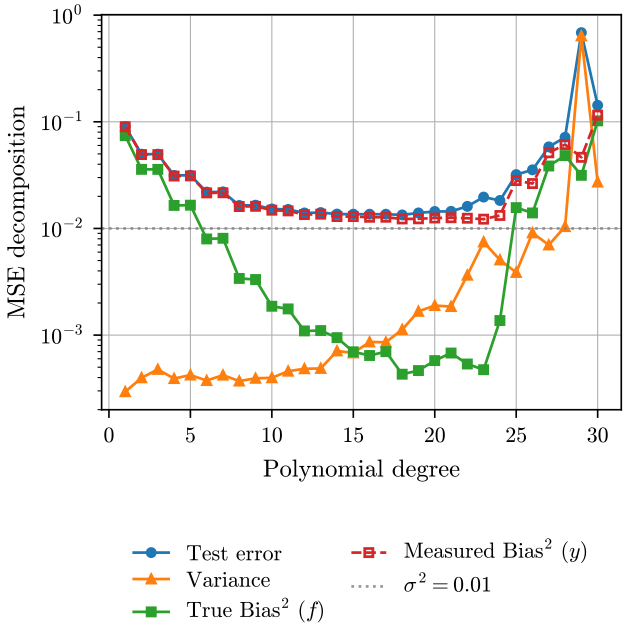


FIG. 7: Bias–variance decomposition of the OLS test error as a function of polynomial degree, estimated using $B = 100$ bootstrap resamples of the training set and evaluated on a fixed test set. The true squared bias is computed relative to the true Runge function, while the measured squared bias is computed relative to the noisy targets y. The dashed horizontal line shows the noise variance σ^2. The data set contained $n = 500$ points with noise level $\sigma = 0.1$.

Although the true function is known in this study, the same analysis was performed using the noisy targets $\boldsymbol{y}$ as reference. The test error and the variance are the same using both references, since only the reference is used to estimate the bias. As described by Eq. (13), the measured squared bias has an irreducible noise contribution. Thus, the true bias of the model may be small, but the measured squared bias has an expected noise floor of σ^2. This is evident in Fig. 7, where the estimated squared bias reduces to approximately σ^2. The choice of reference directly affects the bias–variance balance. Using the noisy targets as reference, the estimated squared bias and variance are comparable at approximately $d \approx 28$, as opposed to $d \approx 15$ when using the true function. This shift of balance is due to the irreducible noise contribution, rather than an increase of the true bias of the model

Fig. 8 investigates the noise contribution in the bias–variance decomposition. It shows that $\text{MSE} - \text{Bias}_f^2 - \text{Var}$ and $\text{Bias}_y^2 - \text{Bias}_f^2$ overlap across all polynomial degrees. This is expected from Eq. (10), which gives $\text{MSE} - \text{Bias}_f^2 - \text{Var} = \sigma^2$, and Eq. (13) which gives $\text{Bias}_y^2 - \text{Bias}_f^2 = \sigma^2$, when computed using the same bootstrap predictions and test set. Deviations from $\sigma^2 = 0.01$ are related to the use of a finite test dataset, such that the averages may differ from their expectation values.

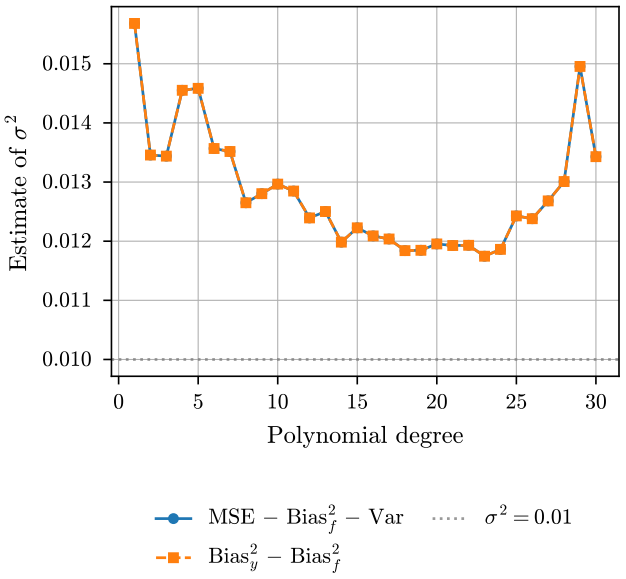


FIG. 8: Numerical verification of the noise variance contribution when using noisy data. The quantities $\text{MSE} - \text{Bias}_f^2 - \text{Var}$ and $\text{Bias}_y^2 - \text{Bias}_f^2$ overlap and remain close to the noise variance $\sigma^2 = 0.01$, as expected.

Finally, the effect of increasing the number of data points in the bias–variance decomposition is shown in Fig. 9 for $n = 50$, 100, and 500 with $\boldsymbol{y}$ as reference. In general, increasing n reduces both the test MSE, bias squared and variance. The estimate of the variance is particularly sensitive to the sample size of the dataset. For $n = 50$, the MSE, variance, and squared bias grows rapidly at high polynomial degrees. This indicates numerical instability related to the small sample size. For high degrees the problem is severely ill-conditioned which leads to unstable predictions. Increasing the sample size to $n = 100$ reduces this behavior, while $n = 500$ exhibits a significantly lower and stable variance. The measured squared bias behaves similarly, with differences of ~ 18 orders of magnitude for $d \gtrsim 25$. The resulting test MSE reflects the same behavior. Thus, increasing the sample size reduces the models sensitivity to a particular training sample for higher-complexity OLS models.

D. Cross–validation for selecting model

Cross-validation (CV) was used to estimate the MSE as a function of polynomial degree for OLS, and as function of both polynomial degree and regularization parameter for Ridge. This was done for $k = 5$ and $k = 10$ in both cases. The degree at which the MSE in minimized is the optimal model complexity.

Optimal model complexity using OLS

Fig. 10 shows the MSE for OLS from CV using $k = 5$ and $k = 10$ folds, together with the MSE obtained from $B = 100$ bootstrap samples. For both values of k, the MSE decreases rapidly at low polynomial degrees until it reaches a broad minimum at intermediate degrees, with the minimum concurring at $d = 18$. For $d \gtrsim 24$, the CV error increases. The MSE obtained with both 5-fold and 10-fold CV exhibit this behavior and pick $d = 18$ as the best complexity. Thus, increasing the number of folds does not affect the best model complexity nor the behavior of the MSE for this case.

Compared to the MSE from 100 bootstrap resamples also shown in Fig. 10, the CV error is generally lower and more stable at higher polynomial degrees. Both methods identify the same region of degrees, with $d = 18$ as the best model complexity for both methods. However, the bootstrap MSE increases rapidly for higher degrees. This differing behavior is due to bootstrap being trained on fewer unique data points, so it has a higher variance and tendency to overfit. In k-fold CV, each observation is used for testing once, while bootstrap resamples training data with replacement, such that some observations are repeated while others are not. Nonetheless, both methods identify the same model complexity.

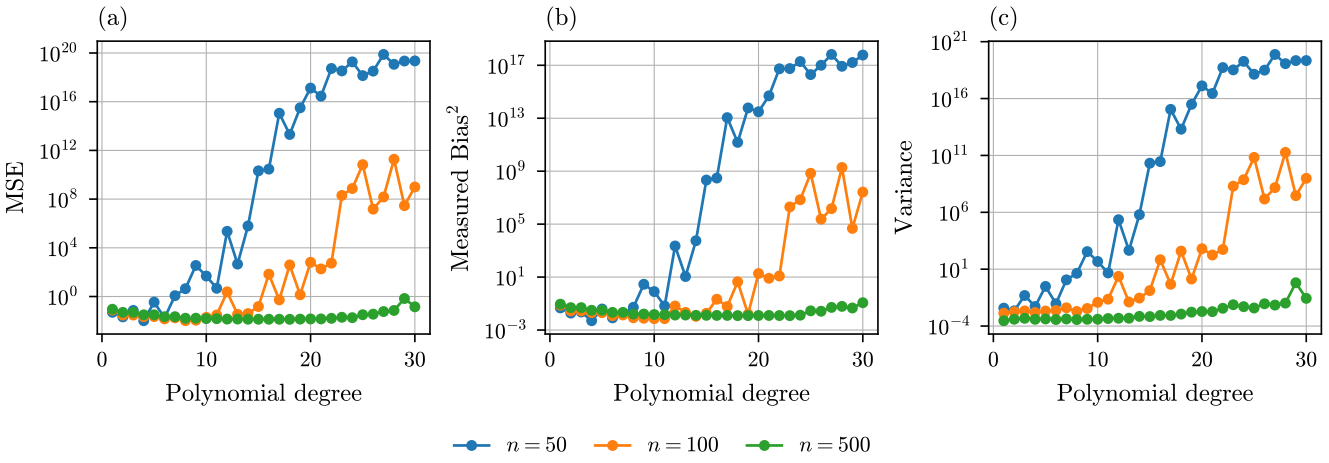


FIG. 9: Bootstrap estimates of (a) test MSE, (b) measured squared bias, and (c) prediction variance for OLS as functions of polynomial degree for $n = 50$, 100, and 500 data points. The estimates were obtained using $B = 100$ bootstrap resamples with noise level $\sigma = 0.1$. The squared bias is measured relative to the noisy targets $\boldsymbol{y}$. Note the logarithmic scale of the vertical axes.

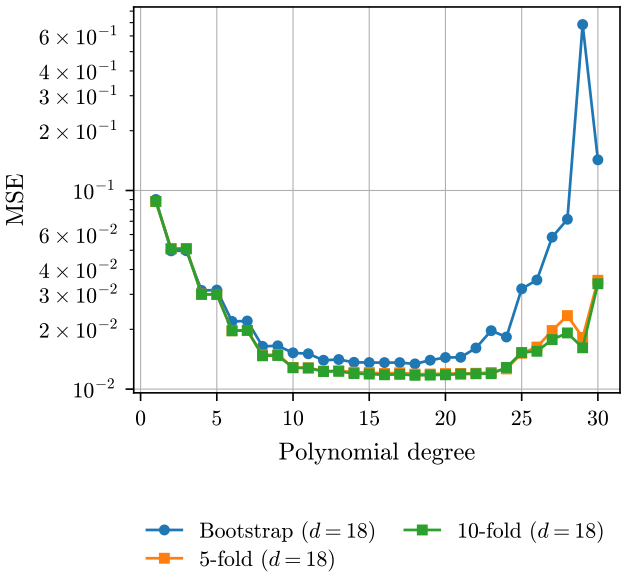


FIG. 10: MSE obtained from CV with $k = 5$ and $k = 10$ for OLS as a function of polynomial degree, together with the MSE obtained from $B = 100$ bootstrap resamples. The dataset contained $n = 500$ points with noise $\sigma = 0.1$. All three experiments attain $d = 18$ as the best model complexity.

Optimal model complexity using Ridge

For Ridge regression, both the regularization parameter and the model complexity must be selected. Fig. 11 shows the MSE obtained from CV and bootstrap as a function of polynomial degree for a fixed $\lambda = 0.01$. This choice of λ is arbitrary and is just to illustrate the stabilizing effect of regularization, rather than being a optimal parameter choice. In all three cases, the MSE decreases rapidly across the first few polynomial degrees before flattening out for $d \gtrsim 8$. The 5-fold and 10-fold MSE are nearly identical, but the 5-fold CV indicates that $d = 30$ is the best degree while 10-fold gives $d = 17$. The bootstrap MSE follows the same pattern, but settles at a slightly higher MSE, with $d = 17$ as the best model complexity. The exact minima are most likely due to random noise and any degree $d \gtrsim 15$ seem to yield comparable MSE. Although the 5-fold CV estimates a much higher optimal complexity, the methods agree on the region of optimal polynomial degrees, even though the exact minima occurred at $d = 30$.

Compared to the OLS results in Fig. 10, the effect of the regularization is evident. For OLS, the bootstrap MSE increased rapidly at higher complexities, with less substantial increase for CV. However, both the bootstrap MSE and the MSE from CV remain stable throughout the higher degrees. This indicates that the Ridge regularization with $\lambda = 0.01$ has a profound impact on the MSE attained from both bootstrap and CV for highly complex models.

Moreover, the influence of λ is shown in Fig. 12 for a fixed polynomial of degree $d = 15$. The shaded regions show the standard deviation of the MSE across folds, which increases as λ increases. For small λ, the MSE is stable, while it starts to increase substantially for approximately $\lambda \gtrsim 10^{-4}$. This is consistent with the parameter shrinkage discussed in Sec. IV B. Further, the MSE from 5-fold and 10-fold CV is nearly identical across the investigated range of λ. Overall, the results show that regularization does not improve the minimum MSE when using CV for

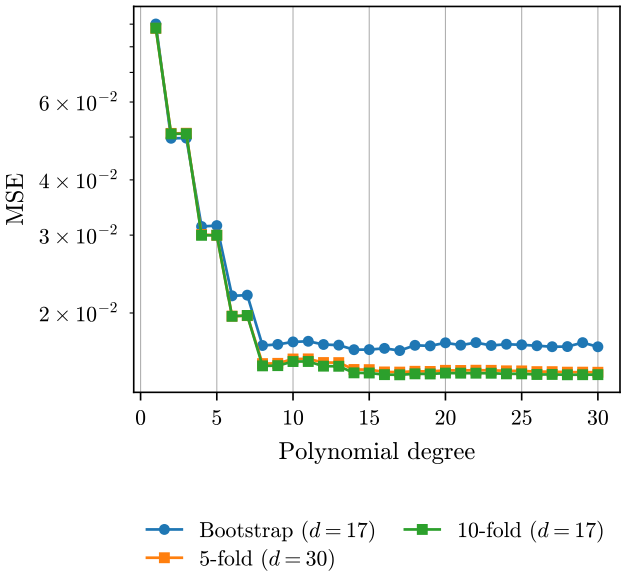


FIG. 11: MSE obtained from CV of polynomial degree, with fixed regularization parameter $\lambda = 0.01$. The MSE obtained from $B = 100$ bootstrap resamples is also shown. The dataset contained $n = 500$ data points with noise $\sigma = 0.1$. The 5-fold CV estimates $d = 30$ as the optimal polynomial degree, while the 10-fold CV and bootstrap agree upon $d = 17$.

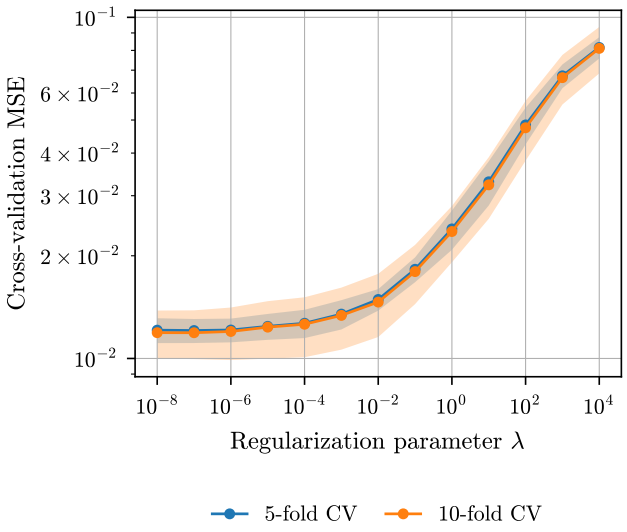


FIG. 12: MSE for Ridge regression as a function of the penalty parameter λ, using 5-fold and 10-fold CV. The polynomial degree was fixed at $d = 15$, and the data had $n = 500$ and noise $\sigma = 0.1$. The shaded regions indicate $\pm$ one standard deviation of the MSE across the cross-validation folds.

$d = 15$, as the error increases as λ increases. However, regularization profoundly improves the stability of highly complex models when using both bootstrap and CV as resampling methods. The effect is most pronounced when using bootstrap resampling, while it gives a small, but notable improvement when using CV.

E. Gradient descent for OLS and Ridge

For our gradient descent experiments, a polynomial of degree $d = 5$ and $n = 100$ data points was used. For Ridge and Lasso, the regularization parameter $\lambda = 1$ was used, which is large enough that Ridge and Lasso differ notably from OLS. These values differ from the optimal values found previously as we measure iterations for comparable cost functions, not necessarily statistical performance. As we want to keep the condition number low, we choose lower polynomial degrees. We do not split the data either as we want to test convergence and not performance against the true solution.

Tbl. V in Appendix C compares gradient descent using analytically derived gradients with gradient descent using automatic differentiation for both OLS and Ridge regression, using a learning rate $\gamma = 0.8\gamma_{\max}$. The analytical gradients were derived from the corresponding cost functions in Secs. II C 1 and II C 2, while automatic differentiation was performed using JAX. The parameters obtained from gradient descent were compared with their closed-form solutions, given by Eqs. (5) and (8). In both cases, the optimization was terminated once the convergence criterion was met, which was set to $\|\boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_t\|_2 < 10^{-8}$.

The analytical and automatically differentiated gradients agree to approximately machine precision. That is, the maximum absolute difference between the gradient components is of order 10^{-16} for both OLS and Ridge. Additionally, both approaches exhibit essentially identical convergence behavior, requiring the same number of iterations and reaching the same relative parameter error with respect to the corresponding closed-form solutions. This confirms that the analytically derived gradients and the JAX implementation are numerically consistent.

Tbl. VI in Appendix C shows how the convergence of OLS and Ridge depends on the learning rate relative to the maximum learning rate, γ/γ_{max}. For small learning rates, gradient descent progresses slowly, and both models reach maximum number of iterations before satisfying the convergence criterion. As the learning rate increases, the number of iterations required for convergence correspondingly decreases. In particular, as γ/γ_{max} approaches unity from below, the number of required iterations notably decreases.

When the learning rate exceeds the stability limit, $\gamma > \gamma_{max}$, gradient descent becomes unstable and diverges. This is clearly observed for $\gamma/\gamma_{max} = 1.01$, where both OLS and Ridge reached the maximum number of it-

erations, and the relative parameter error increases to approximately 10^{83}. The results are therefore consistent with the theoretical stability condition determined, $0 < \gamma < 2/L$, where $L = \mu_{\max}(H)$ is the largest eigenvalue of the Hessian. Hence, the theoretical upper limit is $\gamma_{\max} = 2/\mu_{\max}(H)$.

Ridge regression converges in considerably fewer iterations than OLS. The regularization penalty modifies the Hessian to $H_{\text{Ridge}} = H_{\text{OLS}} + \frac{2\lambda}{n} I$, which shifts all eigenvalues by $2\lambda/n = 0.02$. Thus, the smallest eigenvalues increase, which improves the conditioning of the optimization problem and accelerates convergence. As seen in Tbl. V, the condition number decreases from ~ 777 for OLS to ~ 207 for Ridge, so the Ridge problem is considerably better conditioned. The largest eigenvalue is almost unchanged, so $\gamma_{\max}$ only decreases from 0.3553 to 0.3540. This is consistent with the observed number of iterations required for convergence, which is 5948 for OLS and 1583 for Ridge. The ratio of iterations, ~ 3.8, is close to the ratio of the condition numbers, ~ 3.7.

F. Momentum and adaptive optimization methods

We now wish to study the other iterative methods against OLS and Ridge to compare with plain GD, again without splitting the data. Fig. 14 shows the relative error $||\boldsymbol{\theta}_t - \boldsymbol{\theta}^*||/||\boldsymbol{\theta}^*||$ with respect to the closed-form OLS solution as a function of iterations. The iterations are done by plain gradient descent, momentum, AdaGrad, RMSProp and Adam, for polynomial degree $d = 5$ and $n = 100$ data points. Fig. 15 shows the corresponding results for Ridge regression with $\lambda = 1$. The optimal γ is found by the least iterations needed to reach and stay strictly below the target error, which is considered a converged state. Logarithmically spaced learning rates from 10^{-3} to 10 were checked. Iterations to convergence for all learning rates and all methods are shown in Fig. 13.

Momentum is clearly the fastest converging method, as seen in Figs. 14 and 15. It reaches the target accuracy of 10^{-3} after about 250 iterations for both OLS and Ridge and machine precision after about 1500 iterations. Fluctuations for the remaining iterations are due to floating point rounding. Plain GD with the same learning rate requires about 3000 iterations to reach the target against OLS and is still decreasing during the last iteration. For Ridge, plain GD converges with a relative error of order 10^{-14} after 4000 iterations.

The difference between OLS and Ridge is explained by the condition number of the Hessian. The Ridge penalty shifts all eigenvalues of the Hessian by $2\lambda/n$. Since the convergence is limited by the direction with the smallest eigenvalue, plain GD, momentum and AdaGrad converge considerably faster for Ridge than for OLS.

The learning rate sweep in Fig. 13 shows that the number of iterations scales roughly as $1/\gamma$ for both plain GD and momentum. Momentum scales approximately with 10 compared to plain GD, which is almost exactly what the theory predicts with the effective learning rate scaling as $\gamma/(1-\beta) = 10\gamma$, with $\beta = 0.9$. Neither plain GD or momentum reach convergence for γ's tested above $\gamma_{\max} = 2/\mu_{\max}$, marked with the dotted line. This is shown for GD in Tbl. VI in Appendix C.

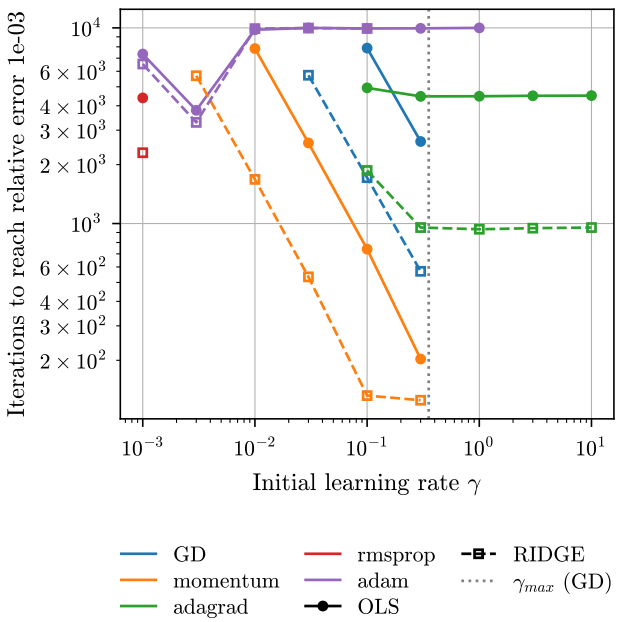


FIG. 13: Number of iterations needed to reach a relative error of 10^{-3} with respect to the closed-form solution, as a function of the initial learning rate γ, for OLS (solid lines, filled markers) and Ridge with $\lambda = 1$ (dashed lines, open markers). The polynomial degree is $d = 5$, with $n = 100$ data points and noise $\sigma = 0.1$. The dotted vertical line marks the stability limit $\gamma_{\max} = 2/\mu_{\max}(H)$. Missing points indicate that the target was not reached, and points at 10^4 correspond to runs that only fell below the target at the iteration limit.

Further, AdaGrad behaves very differently. The number of iterations required to converge is nearly independent of γ, as is evident in Fig. 13. Approximately 5500 iterations are needed for all γs from 0.1 to 10, which is well beyond the GD stability limit, $\gamma_{\max}$. A too large initial γ produces large early gradients that inflate the accumulator $\boldsymbol{r}_t$ and reduce the effective step to $\gamma/\sqrt{\boldsymbol{r}_t}$ until the iteration is stable. As gradients decay, $\boldsymbol{r}_t$ stops growing, and AdaGrad behaves like plain GD with a constant, individually scaled learning rate for each parameter. This gives linear convergence, but at a slower rate than well-tuned GD. For $\gamma \lesssim 0.1$, AdaGrad does not reach the target within the iteration bound. Because $\boldsymbol{r}_t$ is non-decreasing, the effective learning rate becomes non-increasing, so AdaGrad can reduce a too-large γ, but not increase a too-

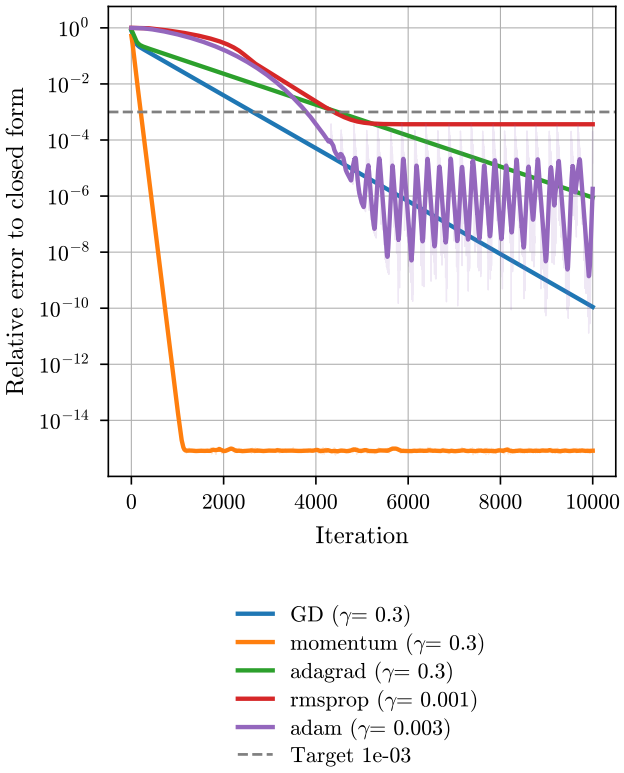


FIG. 14: Relative error $\|\boldsymbol{\theta}_t - \boldsymbol{\theta}^*\|/\|\boldsymbol{\theta}^*\|$ with respect to the closed-form OLS solution as a function of iterations for plain gradient descent and the adaptive methods. The polynomial degree is $d = 5$, with $n = 100$ data points and noise $\sigma = 0.1$. Each method is shown with its best learning rate from Fig. 13.

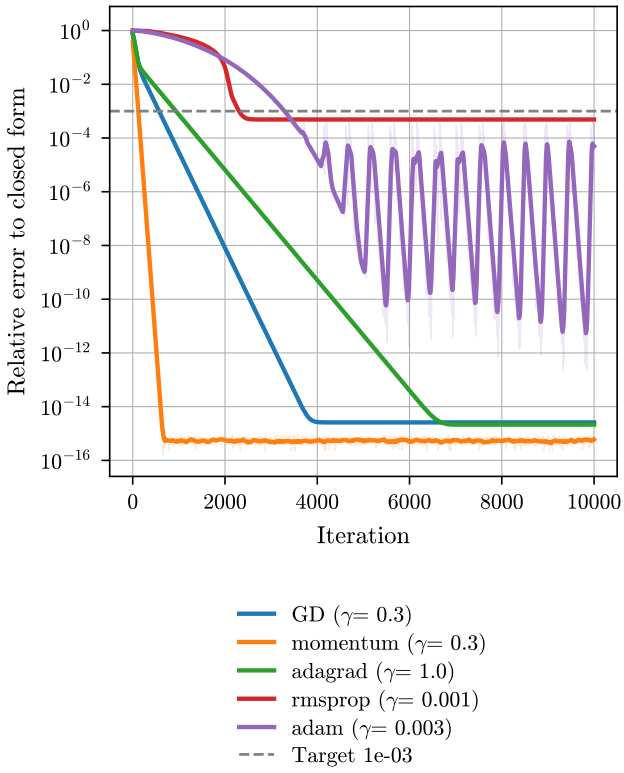


FIG. 15: Relative error $\|\boldsymbol{\theta}_t - \boldsymbol{\theta}^*\|/\|\boldsymbol{\theta}^*\|$ with respect to the closed-form Ridge solution as a function of iterations for plain gradient descent and the adaptive methods. The polynomial degree is $d = 5$, with $n = 100$ data points, noise $\sigma = 0.1$ and $\lambda = 1$. Each method is shown with its best learning rate from Fig. 13.

small one. Regarding plain GD and momentum, AdaGrad benefits from the better conditioning of the Ridge problem, reaching a relative error of order 10^{-14} after about 7000 iterations, as seen in Fig. 15, while the OLS error in Fig. 14 is still of order 10^{-6} at the iteration limit.

RMSProp only reaches the target for the lowest learning rate, as seen in Fig. 13. The error plateaus in the order of 10^{-4} for $\gamma = 10^{-3}$ in Fig. 14 and 15. The reason is that RMSProp normalizes the gradient by a running estimate of its magnitude, so the step length remains of order γ even as the gradient goes to zero. Near the minimum, the iterations oscillate in a band set by γ. We therefore expect the error to decrease with γ, although only one learning rate reached the target in our sweep. A decaying learning rate schedule would also remove this floor.

In Fig. 14 and 15 Adam is one of the slower methods to reach the tolerance, as it reaches the tolerance after approximately 4000 iterations for both OLS and Ridge at $\gamma = 3 \times 10^{-3}$. After reaching the tolerance, Adam's error trend downward, but is interrupted by spikes of order 10^{-4}. These arise when the gradient is small for many consecutive iterations and the second moment $\boldsymbol{v}_t$ becomes small. Then, the effective learning rate $\gamma/\sqrt{\boldsymbol{v}_t}$ grows rapidly, which causes $\boldsymbol{v}_t$ to increase, restoring the stability to the relative error. This continues in a cycle, as clearly shown in Fig. 14 and 15. Fig. 13 shows that Adam converges below the tolerance for many learning rates, but for $\gamma \geq 10^{-2}$, Adam converges at the iteration bound. Therefore, the results become inconclusive due to our definition of convergence, and we can not inspect Adam's behavior beyond the iteration bound.

By the analysis given above, momentum is the most efficient method for a deterministic quadratic problem. Plain GD converge slowly for a small range of γs. AdaGrad is robust against the choice of γ as long as $\gamma \geq 10^{-1}$. RMSProp does not create any convincing results, as only one γ reach the tolerance, and it's relative error is the biggest of all the methods. Adam might be tested in a bad domain as it only reaches and stays below the target for the two smallest initial learning rates.

G. Lasso regression

Unlike OLS and Ridge, Lasso has no closed-form solution, since the ℓ^1 penalty is not differentiable at zero. It therefore tests whether our gradient-based methods can solve a problem without an analytical reference. Fig. 16 shows the relative error with respect to Scikit-Learn's Lasso solution for polynomial degree $d = 5$, $n = 100$ data points, and $\lambda = 1$. For each method, we used the learning rate with the lowest relative error at the maximum number of iterations. Compared to OLS and Ridge, where GD reached low relative error and momentum reached machine precision, all methods against Lasso stall at a noisy floor between 10^{-5} and 10^{-3}. This is expected for subgradient descent with a constant learning rate, since the size of the subgradient step does not decrease as the minimum is approached. Consistent with this, the optimal learning rates for GD and momentum are lower than for OLS and Ridge, as a smaller step gives a lower floor. Momentum again gives the lowest error, while RMSProp gives the highest relative error together with AdaGrad.

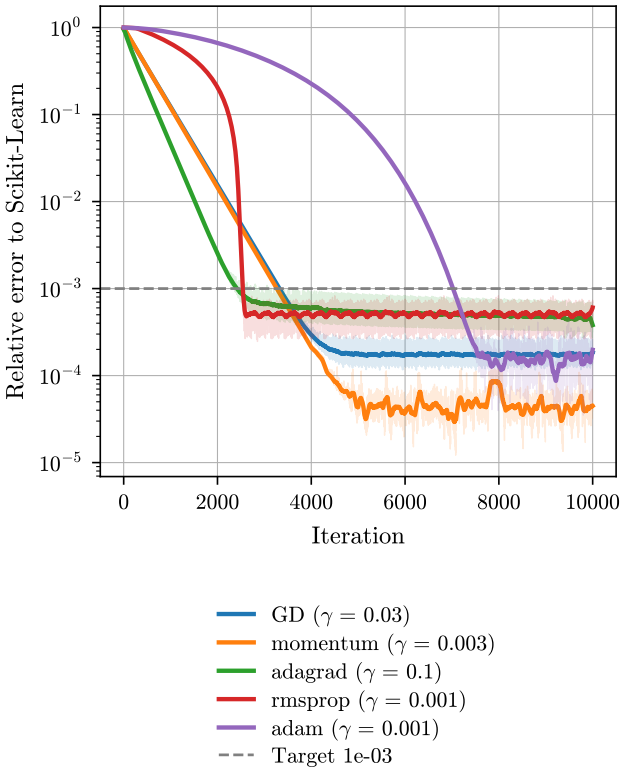


FIG. 16: Relative error to Scikit-Learn's Lasso solution as a function of iterations for plain gradient descent and the adaptive methods. The polynomial degree is $d = 5$, with $n = 100$ data points, noise $\sigma = 0.1$ and $\lambda = 1$. Each method is shown with its best learning rate, given in the legend. The dashed line marks the target relative error of 10^{-3}.

Fig. 17 shows that our solver reproduces the test MSE and R^2 of Scikit-Learn for $\lambda \lesssim 10$, but deviates strongly for $\lambda = 30$ and 100. For large λ, most optimal coefficients are zero, which our solver cannot reproduce.

Moreover, the deviation at large λ is caused by the fixed learning rate. The subgradient term, $(\lambda/n)\,\mathrm{sgn}(\boldsymbol{\theta})$, has a constant magnitude, so coefficients whose optimal value is zero oscillate around zero with an amplitude of order $\gamma\lambda/n$ instead of converging. At $\lambda = 10^2$, this step is larger than the coefficients themselves, and the iterates oscillate between values of order 1. Scikit-Learn instead uses coordinate descent, which minimizes the cost function with respect to one coefficient at a time. Each update is given by a soft-thresholding function that is exactly zero whenever the correlation between the feature and the residual is below the penalty, which allows coefficients to become exactly zero [10]. A decreasing learning rate would reduce the oscillations and allow the coefficients to settle closer to the minimizer.

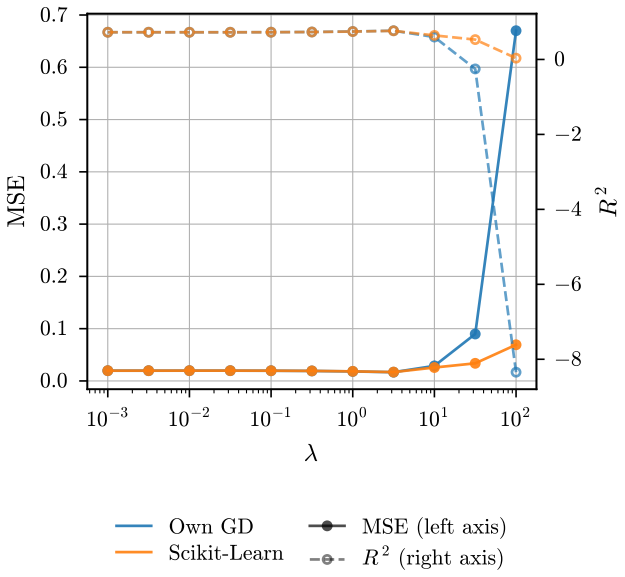


FIG. 17: Test MSE (solid) and R^2 scores (dashed) as a function of the penalty λ for Lasso regression on a logarithmic x-axis, for polynomial degree $d = 5$ and $n = 100$ data points. The orange lines are obtained by Scikit-Learn's Lasso, while the blue are by our own gradient descent solver with fixed learning rate $\gamma = 0.8\gamma_{\max}$.

Gradient descent solvers, such as momentum, solve OLS and Ridge to near machine precision, but Lasso only between 10^{-5} and 10^{-4}. This is sufficient for prediction, as the test MSE agrees with Scikit-Learn for moderate λ, but the sparsity that motivates Lasso is lost as no coefficients become exactly zero. For large λ, the fixed learning rate also prevents convergence, so subgradient descent with a constant learning rate is not a suitable solver for strongly regularized Lasso.

H. Stochastic gradient descent

The effect of varying mini-batch size is shown in Fig. 18 as functions of relative error to the closed-form solution. The relative error after 1000 epochs depends strongly on both the mini-batch size and the optimizer. For the smallest batch-size, GD and momentum diverge, indicating that the fluctuations are too large. Increasing M reduces the stochasticity of the gradient and reduces the number of parameter updates each epoch.

The learning rates used for SGD, listed in Tbl. II, were chosen separately from those stated in Sec. IV F. Learning rates that were effective for full-batch gradient descent did not always remain stable when applied to mini-batches. In particular, momentum gained such large initial fluctuations that the remaining curves became difficult to resolve in the same figure. We therefore selected learning rates that gave stable stochastic optimization for the batch sizes considered.

TABLE II: Initial learning rates used for the stochastic optimization experiments.

Optimizer	Initial learning rate γ
GD	0.1
Momentum	0.03
AdaGrad	1.0
RMSProp	0.003
Adam	0.03

AdaGrad is relatively insensitive to small mini-batch sizes, as it maintains a relative error of $\sim 10^{-2}$, before increasing for larger batches. RMSProp shows the opposite tendency, with the error increasing with the batch size before decreasing again for the full-batch. Adam benefits the most from increasing the batch size. For the full-batch, Adam reaches a relative error of order $\sim 10^{-8}$, while for $M = 10$ the error remains at 10^{-2}. However, as shown in Sec. IV F, the relative error of Adam has a tendency to oscillate and may suffer later error spikes, meaning this should be considered as error at the exact endpoint rather than permanent convergence. Overall, the stochastic gradient noise prevents the parameters from settling exactly at the minimum when a constant learning rate is used. The Ridge solutions show the same qualitative behavior, therefore, the Ridge variants Fig. 18 and 19 are instead found in the Jupyter notebook in the UiO GitHub repository [4]

Fig. 19 compares stochastic optimization with $M = 10$ to the corresponding full-batch methods as a function of epoch. None of the stochastic methods reached the target of 10^{-3} in 1000 epochs. Some SGD methods decreased their initial error faster than their corresponding full-batch method, before the curves eventually crossed.

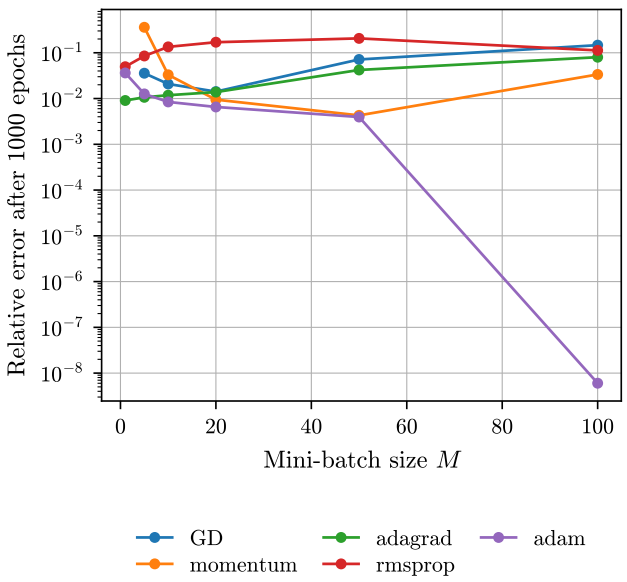


FIG. 18: Relative error to the closed-form OLS solution after 1000 epochs as a function of mini-batch size M for the different optimizers. The full-batch result corresponds to using the entire training set in each update. The learning rates are given in Tbl. II. The corresponding Ridge results show similar behavior and are provided in the Jupyter notebook.

For OLS, Adam was the only full-batch method to reach the target after 589 epochs, and eventually obtained a relative error of 6.02×10^{-9}, as shown in Tbl. VIII. In contrast, stochastic Adam with $M = 10$ reached 8.43×10^{-3} after 1000 epochs. Similar behavior is observed for the other stochastic methods, where the relative error settles approximately between 10^{-2} and 10^{-1}. For $M = 10$, one epoch contains $n/M = 10$ noisy parameter updates, while one full-batch epoch contains a single update using the full gradient.

Fig. 20 displays the effect of introducing an inverse-time-decay learning rate schedule. With a constant learning rate, the stochastic gradient for most models continues to fluctuate near the minimum, preventing the parameters from settling accurately. The decay schedule gradually reduces the learning rate, which can reduce the fluctuations in later epochs. This is especially beneficial for momentum, where the final relative error is reduced compared to the constant learning rate. For this batch size, scheduling increased the error of the remaining methods.

Computationally, stochastic gradient descent provides no advantage for our data set, as shown in Tbl. VIII. Although each mini-batch uses only a fraction of the training data, one epoch requires n/M parameter updates,

[4] https://github.uio.no/victorbj/FYS-STK4155/blob/main/projects/project1/supplementary_figures.ipynb

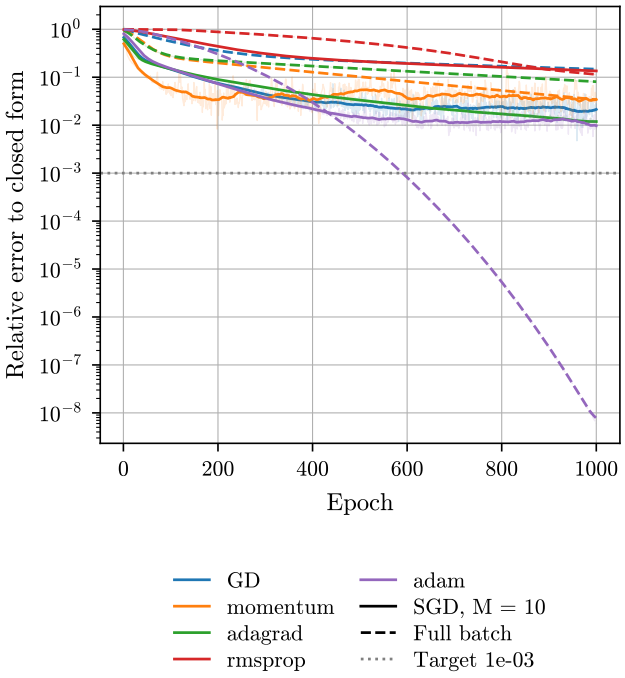


FIG. 19: Convergence of stochastic gradient descent with mini-batch size $M = 10$ compared with full-batch gradient descent for OLS. The relative error is measured with respect to the closed-form solution, and the horizontal line marks the target relative error of 10^{-3}. The bold lines are smoothed with a moving average over $W = 50$ epochs, and the unsmoothed error after each epoch is drawn with lower opacity in the same color. The learning rates are given in Tbl. II. The corresponding Ridge results are provided in the Jupyter notebook.

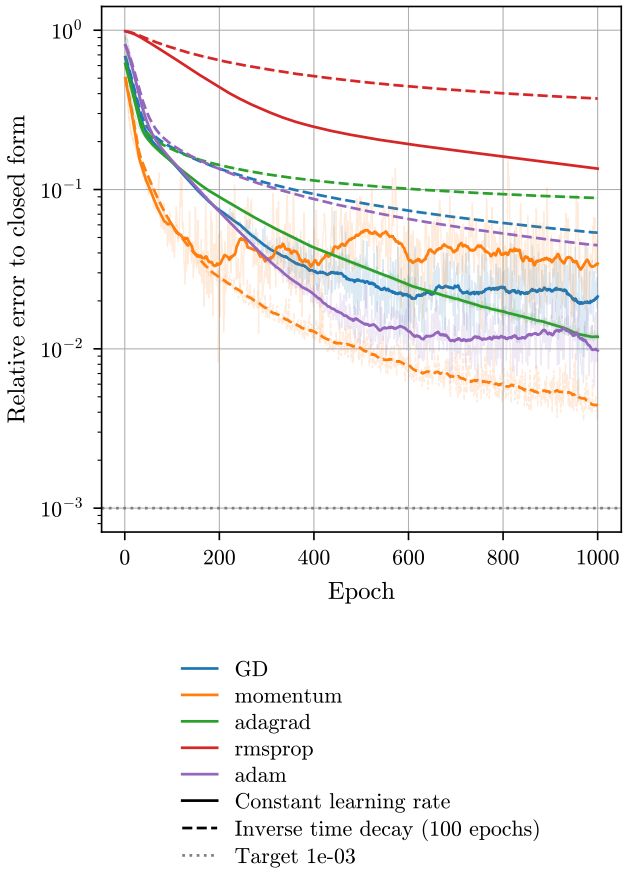


FIG. 20: Relative error to the closed-form OLS solution as a function of epoch for stochastic gradient descent with mini-batch size $M = 10$. Constant learning rates are compared with an inverse-time-decay learning-rate schedule for the different optimizers. The horizontal line marks the target relative error of 10^{-3}. The lines were smoothed with $W = 50$, and the real data can be seen with lower opacity and the same color. The learning rates are given in Tbl. II.

making the overhead call dominate the computation. For $M = 10$, the measured time per epoch for SGD is approximately 0.08 ms, compared to 0.007 ms for full-batch optimization. In this implementation, the additional parameter updates make SGD slower for this small data set. For larger data sets, mini-batching is expected to become more relevant, since evaluating the full gradient at each update becomes more expensive.

The corresponding results using Ridge, given in Tbl. IX and X generally show a similar behavior as OLS. However, Ridge gave noticeably smaller relative errors for several methods for $M = 50$ and the full-batch optimization. For example, the final error for Adam decreases from 6.02×10^{-9} for OLS to machine precision for Ridge. As mentioned in Sec. IV F, Adam may suffer from later error spikes, so this should again be considered as the error at the given endpoint rather than permanent convergence.

Ridge requires marginally more computation per epoch than OLS. However, Fig. 21 shows that the decay schedule is more pronounced for Ridge. All methods except RMSProp obtained a lower final relative error when scheduling was used. In addition, GD and AdaGrad reached the target, while Adam approached it closely. This differs from the case for OLS, where the decay schedule only produced a clear improvement for momentum.

I. Comparison of OLS, Ridge and Lasso

For the final comparison of the regression models, we used $n = 500$ data points with noise $\sigma = 0.1$. The data were split into training and test data. The model selection was performed using 5-fold cross-validation, with the same fold assignment for all methods. OLS was evaluated as function of polynomial degree, whilst Ridge and Lasso as functions of both polynomial degree and regu-

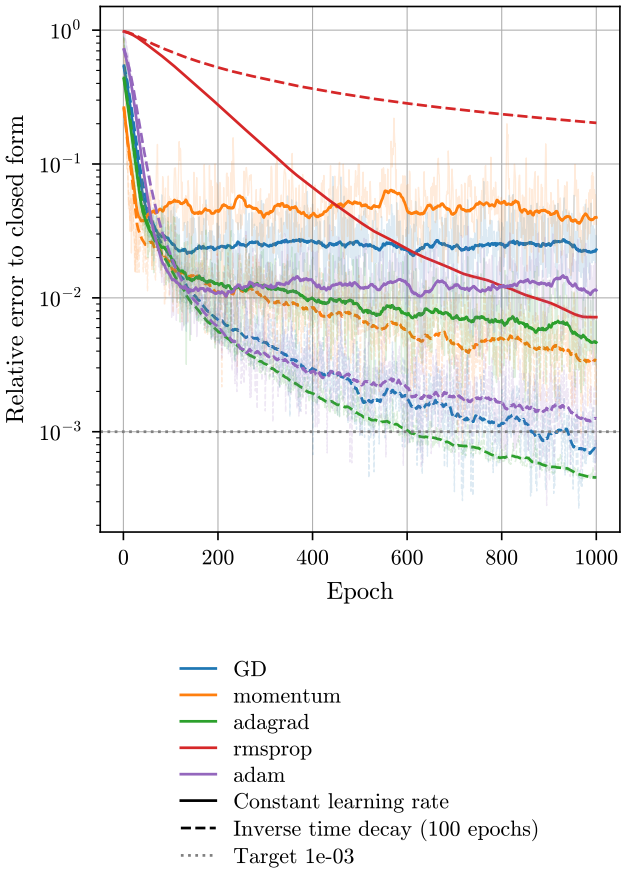


FIG. 21: Relative error to the closed-form Ridge solution as a function of epoch for stochastic gradient descent with mini-batch size $M = 10$. Constant learning rates are compared with an inverse-time-decay learning-rate schedule for the different optimizers. The horizontal line marks the target relative error of 10^{-3}. The lines were smoothed with $W = 50$, and the real data can be seen with lower opacity and the same color. With learning rates in Tbl. II.

larization parameter λ.

Fig. 22 shows the MSE from CV for Ridge regression. As is evident, the MSE is lower for higher degrees and lower λ-values. The lowest MSE was obtained at degree $d = 16$ with penalty $\lambda = 10^{-8}$. However, the MSE is flat over a broad region of the grid, so the exact location of the minimum is most likely due to statistical noise. We generally see that small penalties are preferred, which is consistent with the behavior displayed in Fig. 12. In this region, Ridge is essentially equivalent to OLS.

For Lasso regression solved with our gradient descent implementation, we see in Fig. 23 that the minimal MSE is obtained at degree $d = 10$ with $\lambda = 10^{-4}$. However, this is at the very edge of our grid and in the region where our gradient-descent solver does not fully converge within the chosen iteration limit. Nevertheless, as seen from Fig. 16,

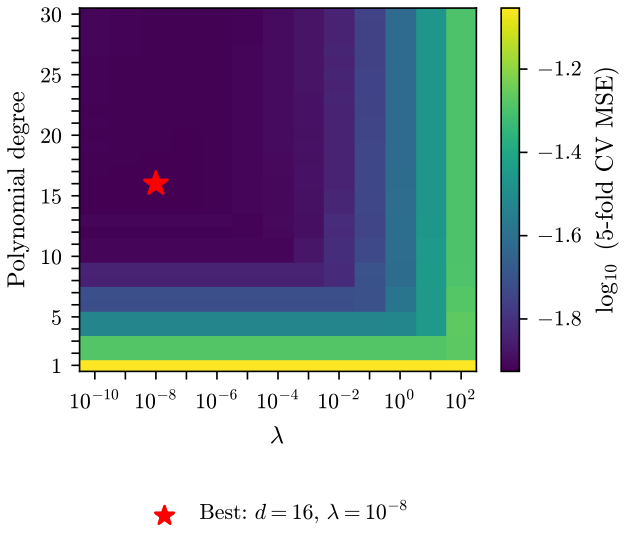


FIG. 22: Five-fold cross-validated MSE of Ridge regression as a function of the regularization parameter λ and the polynomial degree for $n = 400$ data points and noise $\sigma = 0.1$. The color scale shows $\log_{10}$(MSE). The solution is computed in closed form using the SVD of the design matrix. The star marks the lowest MSE.

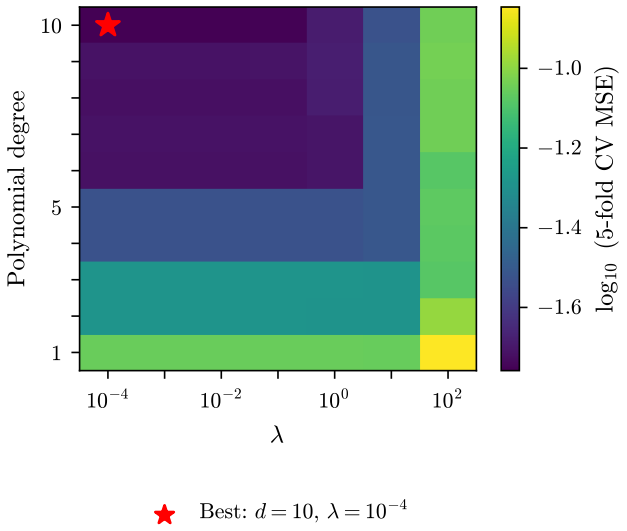


FIG. 23: Five-fold cross-validated MSE of Lasso regression as a function of λ and polynomial degree for $n = 400$ points with noise $\sigma = 0.1$. The color scale shows $\log_{10}$(MSE). The Lasso problem is solved with our gradient descent method with fixed step $0.8\,\gamma_{\max}$, 10^4 iterations, and an analytic subgradient.

the method seems to stagnate before 10,000 iterations. This was only investigated for $d = 5$, which has much better conditioning. The limitations of our solver should therefore be considered when we interpret our Lasso re-

sults.

The best model obtained for each regression method are presented in Tbl. III. OLS and Ridge select neighboring polynomial degrees $d = 15$ and $d = 16$, respectively, with Ridge preferring the weak penalty $\lambda = 10^{-8}$. Their cross-validated MSEs, 0.01188 and 0.01186 differ by far less than fold-to-fold spread and can therefore not be distinguished.

The Lasso models have a lower optimal degree and a higher minimum MSE, with both implementations giving $d = 10$ and $\lambda = 10^{-4}$. Our GD implementation has a cross-validated MSE of 0.01743, which is about 29% above the 0.01348 of Scikit-learn. We believe this is due to non-convergence of our subgradient solver at $d = 10$. Moreover, Lasso was only evaluated for $d \leq 10$, whereas OLS and Ridge reach their minima at $d = 15$–16. As the cross-validated MSE of OLS and Ridge is also still slightly above their minima at $d = 10$, as seen in Fig. 24, the higher Lasso minimum cannot be attributed to Lasso itself, and we cannot conclude that Lasso is necessarily worse for this problem.

TABLE III: Best model for each regression method according to 5-fold cross-validation on the 400 training points. The uncertainty is the standard deviation of the MSE across the folds. The test MSE is obtained by refitting the selected model on all training points and evaluating it once on the 100 held-out test points. The Lasso results are restricted to polynomial degrees ≤ 10 and $\lambda \geq 10^{-4}$, and the minimum lies at the corner of this grid.

Method	Degree	λ	CV MSE	Test MSE
OLS	15	–	0.01188 ± 0.00140	0.01289
Ridge	16	10^{-8}	0.01186 ± 0.00139	0.01266
Lasso (own GD)	10	10^{-4}	0.01743 ± 0.00176	0.01903
Lasso (Scikit-learn)	10	10^{-4}	0.01348 ± 0.00122	0.01527

Fig. 24 shows the cross-validated MSE as function of polynomial degree. OLS and Ridge exhibit approximately the same MSE, with OLS slightly above. This differs from what we see Fig. 10, where OLS gives clear indications of overfitting with increasing MSE for higher degrees. The main difference is the data used, as Fig. 10 uses all 500 points while Fig. 24 uses only the 400 training points. At high degree the design matrix is extremely ill-conditioned, so the OLS solution is sensitive to the data subset. That is, our particular seed could be the reason for this. Further testing with an average over seeds and different data points should be done to investigate this. The stabilizing effect of Ridge at high degree is better seen in the bootstrap results (Tbl. IV). The test MSEs for OLS and Ridge, 0.01289 and 0.01266, are 7–9% above the cross-validated values. This is consistent with a small selection effect.

Both Lasso MSE curves are restricted to $d \leq 10$, but overlaps with the OLS and Ridge errors for lower degrees. This is expected as both the Ridge and Lasso yields lowest MSE for the weak penalties. For $d \gtrsim 6$, the MSE from our implementation deviates from the MSE obtained with the Scikit-learn implementation. The strength of Lasso is to set coefficients to zero, but this does not improve the predictive performance for this problem.

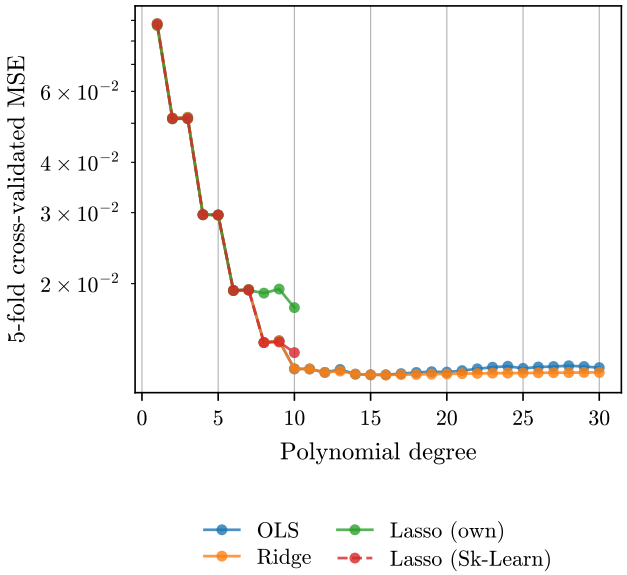


FIG. 24: Five-fold cross-validated MSE as a function of polynomial degree for $n = 400$ data points with noise $\sigma = 0.1$. The same folds are used for all methods and the y-axis is logarithmic. For Ridge and Lasso, the value of λ giving the lowest MSE was used.

We extend our analysis by examining how the choice of regression method affects the bias–variance tradeoff. Tbl. IV shows the squared true bias, variance and test MSE obtained from $B = 50$ bootstrap resamples at selected polynomial degrees. It is evident that the methods act on different terms in the bias–variance decomposition. At $d = 10$, OLS and Ridge give almost identical decompositions and test errors, which is not surprising as the Ridge model is weakly regularized. Increasing the degree to $d = 25$ increases the variance by approximately a factor of 17 for OLS, from 0.00043 to 0.00752. The squared true bias also increases by approximately a factor of 7. Thus, increasing the complexity of an OLS model makes the model sensitive to the particular training sample, as it variance increases substantially.

For Ridge, the variance also increases with polynomial degree, but only by a factor of about 3, compared to the factor of 17 for OLS. At $d = 25$ the squared true bias of Ridge (0.00112) is about 12 times lower than that of OLS (0.01366). It is also lower than at $d = 10$ (0.00193) by a factor of about 1.7, as opposed to OLS where the bias increases with complexity. The test MSE of Ridge at

TABLE IV: Bootstrap bias–variance decomposition with $B = 50$ bootstrap resamples of the 400 training points, evaluated on the 100 test points. Lasso is solved with our own subgradient descent, which is not converged at $d = 10$. Each regularized method uses the λ with the lowest cross-validated MSE at the corresponding degree (cross-validation on the training part). The bias is measured relative to the true function f. The test MSE is the average over the bootstrap models and is therefore not the same quantity as the test MSE in Tbl. III.

Degree	Method	λ	Bias^2	Variance	Test MSE
10	OLS	–	0.00193	0.00043	0.01537
10	Ridge	10^{-6}	0.00193	0.00043	0.01536
10	Lasso	10^{-4}	0.00585	0.00039	0.01972
25	OLS	–	0.01366	0.00752	0.03348
25	Ridge	10^{-6}	0.00112	0.00119	0.01480

$d = 25$ (0.01480) is less than half that of OLS (0.03348). These results indicate that the regularization of Ridge provides a substantially more stable model for higher polynomial degrees than OLS, with lower variance and bias.

Hence, the three regression methods affect the bias–variance tradeoff in different manner. OLS performs well near the optimal polynomial degree, but shows considerable instability as the model complexity increases. In this domain, Ridge performs better, as the prediction error remains stable for highly complex models. The ℓ^2 penalty Ridge offers stabilizes the prediction error and reduces the variance substantially compared to OLS for higher degrees. However, this comes at the cost of additional bias for highly regularized models. Moreover, the ℓ^1 penalty provided by Lasso regularizes models differently by being able to set coefficients exactly to zero. Nevertheless, it is not beneficial for the dataset at our setup generated from Runge's function considered in this study. This is highlighted in Tbl. IV, as Lasso has marginally lower variance than OLS and Ridge, while having higher bias. However, Lasso was not tested for very complex models in this study, such that the results does not define the behavior at higher complexities for Lasso.

Ultimately, our study finds that with 5-fold cross-validation, OLS and Ridge with their optimal parameters are indistinguishable as they have the same approximately the same cross-validated MSE and test MSE. Also the penalty for Ridge is so weak that it essentially becomes OLS. The advantage of Ridge is however that it maintains stability at higher degrees and is the same computational cost as OLS. For our purposes and setup Lasso however falls short. This is due to our restricted parameter search, non-ideal solvers, and a small data set.

V. CONCLUSION

In this study, linear regression methods were applied to Runge's function with added Gaussian noise in order to investigate model complexity, regularization, resampling methods and gradient-based optimization. Ordinary Least Squares, Ridge and Lasso regression were compared in terms of performance. Moreover, bootstrap resampling and cross-validation were used to study the bias–variance tradeoff and select suitable model parameters. In addition, gradient descent and other adaptive optimization methods were implemented and compared to the closed-form solutions of OLS and Ridge. For Lasso, gradient-based optimization was performed and benchmarked with Scikit-learn's coordinate descent solver as reference.

Regularization improved the stability of more complex models, but its benefit depended strongly on the penalty parameter. Ridge reduces the sensitivity of high-degree models, while excessive regularization restricted model complexity and caused underfitting. In the final cross-validation comparison, OLS and Ridge selected neighboring polynomial degrees $d = 15$ and $d = 16$, respectively, and Ridge favored weak regularization $\lambda = 10^{-8}$. Their predictive performance was therefore similar. Lasso obtained a larger cross-validated error and did not provide an advantage for the presented data set. The performance of our own Lasso implementation was limited by the lack of a softening term, causing the parameters to oscillate, instead of converging. However, a small grid, only one seed, and few data points was tested so further research would be beneficial. Overall, the results show that regularization can improve stability, but that a more regularized model does not necessarily provide a better predictive performance.

The gradient descent experiments show that iterative methods can reproduce the closed-form OLS and Ridge solutions when suitable learning rates were used. Analytical gradients and automatic differentiation agreed to approximately machine precision, providing a numerical validation for both methods. The convergence of gradient descent was strongly dependent on the learning rate and agreed with the theoretical stability limit. Increasing the learning rate up to the theoretical limit accelerated convergence, while exceeding it resulted in divergence. Among the optimization methods considered, momentum provided the fastest convergence, reaching machine precision within 1500 iterations. For Lasso, the iterative methods did not reach the same precision, with momentum giving the lowest relative error and oscillating between 10^{-5} and 10^{-4} with respect to the Scikit-learn solution. AdaGrad proved most robust to changes in the learning rate. Overall, the different adaptive methods showed different levels of robustness and convergence behavior.

Stochastic gradient descent introduces an additional

tradeoff between computational cost and fluctuations in the gradient. Small mini-batches produced noisier updates, while some methods experienced unstable behavior. Learning-rate scheduling for OLS only improved the momentum method, while for Ridge most models achieved a lower relative error. Although scheduling improved Ridge, stochastic optimization provided no computational advantage for the relatively small data set considered. In our implementation, the additional parameter updates required within each epoch made SGD slower than the full-batch optimization.

Overall, this work illustrates model complexity, regularization, resampling and gradient-based optimization on a simple noisy function. A natural extension of this work would be to repeat the analysis on functions other than Runge's. In particular, the performance of OLS, Ridge and Lasso may be dependent on the function being approximated. Furthermore, stochastic gradient descent should be investigated on substantially larger data sets and higher-dimensional models, where evaluating the full gradient becomes computationally expensive. It would also be beneficial to run the same experiments over different seeds and look at averages to reduce the possibility of statistical flukes. Such experiments would provide a more appropriate setting determining whether the computational cost of evaluating individual mini-batch updates outweighs the loss in accuracy.

ACKNOWLEDGMENTS

We thank the teaching staff of FYS-STK4155 for helpful feedback and discussions during the group sessions, in the office, and over email. We would also like to thank the coffee at CCSE during the day and the beer at 67 NOK at Chateau Neuf at night for keeping us going. Group 67 signing out and looking forward to new challenges in the next report.

[1] Wikipedia contributors. Machine learning. https://en.wikipedia.org/wiki/Machine_learning, 2026. Accessed: 2026-09-18.

[2] Wikipedia contributors. Runge's phenomenon. https://en.wikipedia.org/wiki/Runge%27s_phenomenon. Accessed: 2026-09-13.

[3] Morten Hjorth-Jensen and collaborators. Fys-stk3155/4155 lecture notes. https://github.com/EducationalMaterialUiO/MachineLearningUiO/blob/main/doc/BookML/Lecturebook.pdf. Accessed: 2026-09-12.

[4] Wikipedia contributors. Hadamard product (matrices). https://en.wikipedia.org/wiki/Hadamard_product_(matrices), 2026. Accessed: 2026-09-23.

[5] PyTorch contributors. Adagrad — pytorch documentation. https://docs.pytorch.org/docs/2.12/generated/torch.optim.Adagrad.html, 2026. Accessed: 2026-09-23.

[6] PyTorch contributors. Rmsprop — pytorch documentation. https://docs.pytorch.org/docs/stable/generated/torch.optim.RMSprop.html, 2026. Accessed: 2026-09-23.

[7] PyTorch contributors. Adam — pytorch documentation. https://docs.pytorch.org/docs/main/generated/torch.optim.Adam.html, 2026. Accessed: 2026-09-23.

[8] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585:357–362, 2020.

[9] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12(85):2825–2830, 2011.

[10] Scikit-learn developers. Linear models: Lasso — scikit-learn user guide. https://scikit-learn.org/stable/modules/linear_model.html#lasso, 2026. Accessed: 2026-10-02.

[11] SciPy contributors. scipy.ndimage.uniform_filter1d — scipy documentation. https://docs.scipy.org/doc/scipy/reference/generated/scipy.ndimage.uniform_filter1d.html, 2026. Accessed: 2026-09-30.

[12] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, 17(3):261–272, 2020.

[13] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Yash Katariya, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018.

[14] John D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.

[15] Yurii Nesterov. *Introductory Lectures on Convex Optimization: A Basic Course*, volume 87 of *Applied Optimization*. Kluwer Academic Publishers, 2004.

Appendix A: Derivation of bias–variance decomposition

The observed target value for a data set $\mathbf{x} = \{x_j\}_{j=0}^{n-1}$ is

$$\boldsymbol{y} = \mathbf{f} + \boldsymbol{\varepsilon}, \qquad \boldsymbol{\varepsilon}_i \sim \mathcal{N}(0, \sigma^2), \tag{A1}$$

such that $\mathbb{E}[\boldsymbol{\varepsilon}] = 0$ and $\mathbb{E}[\boldsymbol{\varepsilon}^2] = \sigma^2$. We define the error as the observed target values minus the model predictions. The expected mean squared prediction error is

$$\mathbb{E}\left[(\boldsymbol{y} - \tilde{\boldsymbol{y}})^2\right] = \frac{1}{n}\sum_{1=0}^{n-1}(y_i - \tilde{y}_i)^2$$

Inserting Eq. (A1) into this expression, we obtain

$$\mathbb{E}\left[(\boldsymbol{y} - \tilde{\boldsymbol{y}})^2\right] = \mathbb{E}\left[(\boldsymbol{f} + \boldsymbol{\varepsilon} - \tilde{\boldsymbol{y}})^2\right].$$

Adding and subtracting $\mathbb{E}[\tilde{\boldsymbol{y}}]$ inside the square on the right hand side gives

$$\mathbb{E}\left[(\boldsymbol{y}-\tilde{\boldsymbol{y}})^2\right] = \mathbb{E}\left[(\boldsymbol{f}+\boldsymbol{\varepsilon}-\tilde{\boldsymbol{y}}+\mathbb{E}\left[\tilde{\boldsymbol{y}}\right]+\mathbb{E}\left[\tilde{\boldsymbol{y}}\right])^2\right]. \tag{A2}$$

To simplify notation, we define

$$A = \boldsymbol{f} - \mathbb{E}[\tilde{\boldsymbol{y}}], \qquad B = \mathbb{E}[\tilde{\boldsymbol{y}}] - \tilde{\boldsymbol{y}}, \qquad C = \boldsymbol{\varepsilon},$$

where A is deterministic for a fixed set of inputs, and the noise C is assumed to be independent of the fitted prediction and therefore of B. Substituting this into Eq. (A2) simplifies the expression to

$$\begin{aligned}\mathbb{E}\left[(\boldsymbol{y}-\tilde{\boldsymbol{y}})^2\right] &= \mathbb{E}\left[(A+B+C)^2\right] \\ &= \mathbb{E}[A^2] + \mathbb{E}[B^2] + \mathbb{E}[C^2] + 2\mathbb{E}[AB] + 2\mathbb{E}[AC] + 2\mathbb{E}[BC].\end{aligned} \tag{A3}$$

Now we compute each term separately. Since A is deterministic for a given x_i, the expectation value of A^2 is the expression itself,

$$\mathbb{E}[A^2] = \left(\boldsymbol{f} - \mathbb{E}[\tilde{\boldsymbol{y}}]\right)^2.$$

Furthermore, we have

$$\mathbb{E}[B^2] = \mathbb{E}\left[\left(\tilde{\boldsymbol{y}} - \mathbb{E}[\tilde{\boldsymbol{y}}]\right)^2\right] = \mathrm{Var}(\tilde{\boldsymbol{y}}),$$

since $\mathrm{Var}(X) = \mathbb{E}\left[(X - \mathbb{E}\left[X\right])^2\right]$ by definition. The next term is simply

$$\mathbb{E}[C^2] = \mathbb{E}[\boldsymbol{\varepsilon}^2] = \sigma^2,$$

due to the definition of $\boldsymbol{\varepsilon}$. For the cross terms, we use that $\mathbb{E}\left[XY\right] = \mathbb{E}\left[X\right]\mathbb{E}\left[Y\right]$ for independent variables X and Y. Thus,

$$\mathbb{E}[AB] = \mathbb{E}\left[\left(\boldsymbol{f} - \mathbb{E}[\tilde{\boldsymbol{y}}]\right)\left(\mathbb{E}[\tilde{\boldsymbol{y}}] - \tilde{\boldsymbol{y}}\right)\right].$$

Since $\boldsymbol{f} - \mathbb{E}[\tilde{\boldsymbol{y}}]$ is not stochastic, it can be taken outside the expectation value,

$$\mathbb{E}[AB] = \left(\boldsymbol{f} - \mathbb{E}[\tilde{\boldsymbol{y}}]\right)\mathbb{E}\left[\mathbb{E}[\tilde{\boldsymbol{y}} - \tilde{\boldsymbol{y}}] = 0,\right.$$

because $\mathbb{E}\left[\mathbb{E}\left[\tilde{\boldsymbol{y}}\right] - \tilde{\boldsymbol{y}}\right] = \mathbb{E}\left[\tilde{\boldsymbol{y}}\right] - \mathbb{E}\left[\tilde{\boldsymbol{y}}\right] = 0$. In the same manner,

$$\mathbb{E}[AC] = \left(\boldsymbol{f} - \mathbb{E}[\tilde{\boldsymbol{y}}]\right)\mathbb{E}[\boldsymbol{\varepsilon}] = 0,$$

since $\mathbb{E}[\boldsymbol{\varepsilon}] = 0$ by definition. At last, the final term is

$$\mathbb{E}[BC] = \mathbb{E}\left[\left(\mathbb{E}[\tilde{\boldsymbol{y}}] - \tilde{\boldsymbol{y}}\right)\boldsymbol{\varepsilon}\right] = 0,$$

because the noise has zero mean and is assumed to be independent of the training set. Putting all the terms together into Eq. (A3) and averaging over the n data points gives

$$\mathbb{E}\left[(\boldsymbol{y}-\tilde{\boldsymbol{y}})^2\right] = \frac{1}{n}\sum_{i=1}^{n}\left(f(x_i) - \mathbb{E}[\tilde{\boldsymbol{y}}]\right)^2 + \frac{1}{n}\sum_{i=1}^{n}\left(\tilde{y}_i - \mathbb{E}[\tilde{\boldsymbol{y}}]\right)^2 + \sigma^2. \tag{A4}$$

Defining the bias of the model as

$$\text{Bias}^2[\tilde{\boldsymbol{y}}] = \frac{1}{n}\sum_{i=1}^{n}\left(f(x_i) - \mathbb{E}[\tilde{\boldsymbol{y}}]\right)^2 = \mathbb{E}\left[\left(\boldsymbol{f} - \mathbb{E}\left[\tilde{\boldsymbol{y}}\right]\right)^2\right], \tag{A5}$$

and the variance as

$$\text{Var}[\tilde{\boldsymbol{y}}] = \frac{1}{n}\sum_{i=1}^{n}\left(\tilde{y}_i - \mathbb{E}[\tilde{\boldsymbol{y}}]\right)^2 = \mathbb{E}\left[\left(\tilde{\boldsymbol{y}} - \mathbb{E}\left[\tilde{\boldsymbol{y}}\right]\right)^2\right].$$

Then Eq. (A4) is reduced to

$$\mathbb{E}\left[(\boldsymbol{y} - \tilde{\boldsymbol{y}})^2\right] = \text{Bias}^2[\tilde{\boldsymbol{y}}] + \text{Var}[\tilde{\boldsymbol{y}}] + \sigma^2,$$

where the squared vector difference is meant as the mean squared component wise error.

When the underlying function $\boldsymbol{f}$ is unknown, we use the observed target values $\boldsymbol{y}$ instead. Inserting this in Eq. (A5) gives

$$\text{Bias}^2_{\text{est.}}[\tilde{\boldsymbol{y}}] = \mathbb{E}\left[\left(\boldsymbol{y} - \mathbb{E}[\tilde{\boldsymbol{y}}]\right)^2\right],$$

where the subscript est. stands for estimate. Using $\boldsymbol{y} = \boldsymbol{f} + \boldsymbol{\varepsilon}$, this becomes

$$\begin{aligned}\text{Bias}^2_{\text{est.}}[\tilde{\boldsymbol{y}}] &= \mathbb{E}\left[\left(\boldsymbol{f} + \boldsymbol{\varepsilon} - \mathbb{E}[\tilde{\boldsymbol{y}}]\right)^2\right] \\ &= \mathbb{E}\left[\left(\boldsymbol{f} - \mathbb{E}\left[\tilde{\boldsymbol{y}}\right]\right)^2\right] + 2\mathbb{E}\left[\left(\boldsymbol{f} - \mathbb{E}\left[\tilde{\boldsymbol{y}}\right]\right)\boldsymbol{\varepsilon}\right] + \mathbb{E}\left[\boldsymbol{\varepsilon}^2\right].\end{aligned}$$

Recognizing the first term as the definition in Eq. (11) and utilizing that $\mathbb{E}\left[\boldsymbol{\varepsilon}^2\right] = \sigma^2$ and $\mathbb{E}\left[\boldsymbol{\varepsilon}\right] = 0$, we obtain

$$\text{Bias}^2_{\text{est.}}[\tilde{\boldsymbol{y}}] = \text{Bias}^2[\tilde{\boldsymbol{y}}] + \sigma^2,$$

where the estimated bias is averaged over different realizations of the test-noise.

Appendix B: Proof of convergence of gradient descent

In this appendix, we try to rigorously prove the convergence of the gradient descent method. We will throughout this section denote, for ease of notation, the cost function $C(\boldsymbol{\theta}) \equiv C(\boldsymbol{X}, \boldsymbol{\theta})$ and $\nabla_{\boldsymbol{\theta}} \equiv \nabla$. We will also need that the cost function, $C(\boldsymbol{\theta})$, is convex, i.e.

$$C\big(\lambda\boldsymbol{\theta}_1 + (1-\lambda)\boldsymbol{\theta}_2\big) \leq \lambda C(\boldsymbol{\theta}_1) + (1-\lambda)C(\boldsymbol{\theta}_2), \qquad \forall \boldsymbol{\theta}_1, \boldsymbol{\theta}_2, \forall \lambda \in [0,1].$$

If the cost function, $C(\boldsymbol{\theta})$, is differentiable, it is convex if and only if its domain D_C is convex and

$$C(\boldsymbol{\theta}_2) \geq C(\boldsymbol{\theta}_1) + \nabla C(\boldsymbol{\theta}_1)^T(\boldsymbol{\theta}_2 - \boldsymbol{\theta}_1), \tag{B1}$$

holds for all $\boldsymbol{\theta}_1, \boldsymbol{\theta}_2$. We then consider the gradient descent update

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \gamma \nabla C(\boldsymbol{\theta}_k), \qquad \gamma > 0. \tag{B2}$$

We now show that $C(\boldsymbol{\theta}_k) \to_{k\to\infty} C^*$, and that $\boldsymbol{\theta}_k \to_{k\to\infty} \bar{\boldsymbol{\theta}}$ which is a global minimizer for C, giving $C(\bar{\boldsymbol{\theta}}) = C^*$. Firstly, we introduce some lemmas.

Lemma 1: [Descent Lemma][5] Let $C : \mathbb{R}^p \to \mathbb{R}$ be differentiable with Lipschitz-continuous derivative with constant L.

$$C(\boldsymbol{\theta} + \boldsymbol{d}) \leq C(\boldsymbol{\theta}) + \nabla C(\boldsymbol{\theta})^T \boldsymbol{d} + \frac{L}{2}\|\boldsymbol{d}\|_2^2, \qquad \forall \boldsymbol{\theta}, \boldsymbol{d} \in \mathbb{R}^p. \tag{B3}$$

Proof of Descent Lemma 1: The fundamental theorem of calculus gives

$$C(\boldsymbol{\theta} + \boldsymbol{d}) - C(\boldsymbol{\theta}) = \int_0^1 \nabla C(\boldsymbol{\theta} + t\boldsymbol{d})^T \boldsymbol{d}\, dt.$$

We then subtract and add $\nabla C(\boldsymbol{\theta})^T \boldsymbol{d}$ under the integral sign to get

$$C(\boldsymbol{\theta} + \boldsymbol{d}) - C(\boldsymbol{\theta}) = \nabla C(\boldsymbol{\theta})^T \boldsymbol{d} + \int_0^1 \left(\nabla C(\boldsymbol{\theta} + t\boldsymbol{d}) - \nabla C(\boldsymbol{\theta})\right)^T \boldsymbol{d}\, dt.$$

By the Cauchy–Schwarz inequality, for each $t \in [0,1]$,

$$\left(\nabla C(\boldsymbol{\theta} + t\boldsymbol{d}) - \nabla C(\boldsymbol{\theta})\right)^T \boldsymbol{d} \leq \|\nabla C(\boldsymbol{\theta} + t\boldsymbol{d}) - \nabla C(\boldsymbol{\theta})\|_2 \, \|\boldsymbol{d}\|_2,$$

and by L-Lipschitz continuity of ∇C, since $\|(\boldsymbol{\theta} + t\boldsymbol{d}) - \boldsymbol{\theta}\|_2 = t\|\boldsymbol{d}\|_2$,

$$\|\nabla C(\boldsymbol{\theta} + t\boldsymbol{d}) - \nabla C(\boldsymbol{\theta})\|_2 \leq Lt\|\boldsymbol{d}\|_2.$$

Combining the two bounds,

$$\left(\nabla C(\boldsymbol{\theta} + t\boldsymbol{d}) - \nabla C(\boldsymbol{\theta})\right)^T \boldsymbol{d} \leq Lt\|\boldsymbol{d}\|_2^2.$$

Integrating over $t \in [1,0]$,

$$\int_0^1 \left(\nabla C(\boldsymbol{\theta} + t\boldsymbol{d}) - \nabla C(\boldsymbol{\theta})\right)^T \boldsymbol{d}\, dt \;\leq\; \|\boldsymbol{d}\|_2^2 \int_0^1 Lt\, dt = \frac{L}{2}\|\boldsymbol{d}\|_2^2.$$

Therefore

$$C(\boldsymbol{\theta} + \boldsymbol{d}) - C(\boldsymbol{\theta}) - \nabla C(\boldsymbol{\theta})^T \boldsymbol{d} \;\leq\; \frac{L}{2}\|\boldsymbol{d}\|_2^2.$$

■

[5] Inspiration of the statement and proof from [3, Eq. 4.24]

Lemma 2: [Co-Coercivity][6]

Assume C is a convex function with a Lipschitz continuous gradient, with Lipschitz constant L on $\mathbb{R}^p$. Then the following inequality holds for all $\boldsymbol{\theta}, \boldsymbol{\theta}' \in \mathbb{R}^p$.

$$\frac{1}{L}\|\nabla C(\boldsymbol{\theta}) - \nabla C(\boldsymbol{\theta}')\|_2^2 \leq (\nabla C(\boldsymbol{\theta}) - \nabla C(\boldsymbol{\theta}'))^T (\boldsymbol{\theta} - \boldsymbol{\theta}'). \tag{B4}$$

Proof of Co-Coercivity Lemma 2: We fix $\boldsymbol{\theta}' \in \mathbb{R}^p$ and consider the function

$$\phi(\boldsymbol{z}) \equiv C(\boldsymbol{z}) - (\nabla C(\boldsymbol{\theta}'))^T \boldsymbol{z}.$$

Then both C and ϕ are convex and $\nabla\phi = \nabla C - \nabla C(\boldsymbol{\theta}')$ is L-Lipschitz. Further $\nabla\phi(\boldsymbol{\theta}') = 0$, so $\boldsymbol{\theta}'$ is a global minimizer. As $\boldsymbol{\theta}'$ is a global minimizer, the Descent Lemma (B3) gives

$$\phi(\boldsymbol{\theta}') \;\leq\; \phi\left(\boldsymbol{z} - \frac{1}{L}\nabla\phi(\boldsymbol{z})\right) \;\leq\; \phi(\boldsymbol{z}) - \frac{1}{2L}\|\nabla\phi(\boldsymbol{z})\|_2^2.$$

Setting $\boldsymbol{z} = \boldsymbol{\theta}$ and expanding ϕ gives

$$C(\boldsymbol{\theta}') - \nabla C(\boldsymbol{\theta}')^T\boldsymbol{\theta}' \;\leq\; C(\boldsymbol{\theta}) - \nabla C(\boldsymbol{\theta}')^T\boldsymbol{\theta} - \frac{1}{2L}\|\nabla C(\boldsymbol{\theta}) - \nabla C(\boldsymbol{\theta}')\|_2^2,$$

giving

$$\frac{1}{2L}\|\nabla C(\boldsymbol{\theta}) - \nabla C(\boldsymbol{\theta}')\|_2^2 \leq C(\boldsymbol{\theta}) - C(\boldsymbol{\theta}') - \nabla C(\boldsymbol{\theta}')^T(\boldsymbol{\theta} - \boldsymbol{\theta}').$$

Switching the role of $\boldsymbol{\theta}$ and $\boldsymbol{\theta}'$, we get

$$\frac{1}{2L}\|\nabla C(\boldsymbol{\theta}) - \nabla C(\boldsymbol{\theta}')\|_2^2 \leq C(\boldsymbol{\theta}') - C(\boldsymbol{\theta}) - \nabla C(\boldsymbol{\theta})^T(\boldsymbol{\theta}' - \boldsymbol{\theta}).$$

Adding the inequalites $C(\boldsymbol{\theta}) - C(\boldsymbol{\theta}')$ cancel, and we get

$$\frac{1}{L}\|\nabla C(\boldsymbol{\theta}) - \nabla C(\boldsymbol{\theta}')\|_2^2 \leq (\nabla C(\boldsymbol{\theta}) - \nabla C(\boldsymbol{\theta}'))^T (\boldsymbol{\theta} - \boldsymbol{\theta}').$$

■

This gives us our main claim about convergence.

Claim 1: [7] Assume C is a convex function on $\mathbb{R}^p$ where its gradient is Lipschitz continuous with constant L,

$$\|\nabla C(\boldsymbol{\theta}) - \nabla C(\boldsymbol{\theta}')\|_2 \leq L\|\boldsymbol{\theta} - \boldsymbol{\theta}'\|_2, \qquad \forall \boldsymbol{\theta}, \boldsymbol{\theta}' \in \mathbb{R}^p.$$

Also let C be bounded below and attain its infimum at some point $\boldsymbol{\theta}^*$. Then for any step size $\gamma \in (0, 2/L)$, the gradient descent update from Eq. (B2) produces a sequence $(C(\boldsymbol{\theta}_k))_k$ such that $C_k \to_{k\to\infty} C^*$, where $C^* \equiv \inf_{\boldsymbol{\theta}\in\mathbb{R}^p} C(\boldsymbol{\theta})$ and $\boldsymbol{\theta}_k$ converges to a point $\bar{\boldsymbol{\theta}} \in \mathbb{R}^p$ which is a global minimizer of C such that $C(\bar{\boldsymbol{\theta}}) = C^*$.

Proof of Claim 1: The inital steps on the proof is based on proofs found in [15, Section 1.2.3 and Theorem 2.1.14]

Firstly, our domain, $\mathbb{R}^p$, is convex. As the gradient $\nabla C(\boldsymbol{\theta})$ is Lipschitz continuous, the descent lemma, with $\boldsymbol{d} = -\gamma\nabla C(\theta_k)$, gives

$$C(\boldsymbol{\theta}_{k+1}) \leq C(\boldsymbol{\theta}_k) - \gamma\nabla C(\boldsymbol{\theta}_k)^T\nabla C(\boldsymbol{\theta}_k) + \frac{L\gamma^2}{2}\|\nabla C(\boldsymbol{\theta}_k)\|_2^2.$$

---

[6] This is also known as Baillon–Haddad theorem. Inspiration of the statement and proof from [15, Lemma 2.1.5]

[7] Expanding on [3, Theorem4.1]

Defining $c(\gamma) \equiv \gamma\left(1 - \frac{L\gamma}{2}\right)$

$$C(\boldsymbol{\theta}_{k+1}) \leq C(\boldsymbol{\theta}_k) - c(\gamma)\ \|\nabla C(\boldsymbol{\theta}_k)\|_2^2,$$

so C is non-increasing for positive $c(\gamma)$, meaning $\gamma \in (0, 2/L)$. This can be rewritten as

$$c(\gamma)\|\nabla C(\boldsymbol{\theta}_k)\|_2^2 \ \leq\ C(\boldsymbol{\theta}_k) - C(\boldsymbol{\theta}_{k+1}).$$

We then take a telescope sum from $k = 0$ to $N - 1$, giving

$$c(\gamma)\sum_{k=0}^{N-1} \|\nabla C(\boldsymbol{\theta}_k)\|_2^2 \ \leq\ C(\boldsymbol{\theta}_0) - C(\boldsymbol{\theta}_N) \ \leq\ C(\boldsymbol{\theta}_0) - C^* < \infty,$$

meaning that $\nabla C(\boldsymbol{\theta}_k) \to 0$ as $\sum_k \|\nabla C(\boldsymbol{\theta}_k)\|_2^2$ is bounded. Further, from Equation (B4), using that $\nabla C(\boldsymbol{\theta}^*) = 0$, we get

$$\frac{1}{L}\|\nabla C(\boldsymbol{\theta}_k)\|_2^2 \ \leq (\nabla C(\boldsymbol{\theta}_k))^T(\boldsymbol{\theta}_k - \boldsymbol{\theta}^*).$$

Inserting this in the gradient descent update (B2) and subtracting by $\boldsymbol{\theta}^*$ on both sides, we get

$$\begin{aligned}
\|\boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}^*\|_2^2 &= \|(\boldsymbol{\theta}_k - \boldsymbol{\theta}^*) - \gamma\nabla C(\boldsymbol{\theta}_k)\|_2^2 \\
&= \|\boldsymbol{\theta}_k - \boldsymbol{\theta}^*\|_2^2 - 2\gamma\nabla C(\boldsymbol{\theta}_k)^T(\boldsymbol{\theta}_k - \boldsymbol{\theta}^*) + \gamma^2\|\nabla C(\boldsymbol{\theta}_k)\|_2^2 \\
&\leq \|\boldsymbol{\theta}_k - \boldsymbol{\theta}^*\|_2^2 - 2\gamma\frac{1}{L}\|\nabla C(\boldsymbol{\theta}_k)\|_2^2 + \gamma^2\|\nabla C(\boldsymbol{\theta}_k)\|_2^2 \\
&= \|\boldsymbol{\theta}_k - \boldsymbol{\theta}^*\|_2^2 - \gamma\left(\frac{2}{L} - \gamma\right)\|\nabla C(\boldsymbol{\theta}_k)\|_2^2.
\end{aligned}$$

Assuming $\gamma \in (0, 2/L)$, we get that $(\boldsymbol{\theta}_k)_k$ is bounded

$$\|\boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}^*\|_2^2 \leq \|\boldsymbol{\theta}_k - \boldsymbol{\theta}^*\|_2^2.$$

As $(\boldsymbol{\theta}_k)_k$ is bounded on $\mathbb{R}^p$, by Bolzano–Weierstrass, there exists a convergent subsequence $(\boldsymbol{\theta}_j)_j$ converging to an element $\bar{\boldsymbol{\theta}} \in \mathbb{R}^p$. As ∇C is continuous,

$$\lim_{j\to\infty} \nabla C(\boldsymbol{\theta}_j) = \nabla C(\lim_{j\to\infty} \boldsymbol{\theta}_j) = \nabla C(\bar{\boldsymbol{\theta}}).$$

However, as $\nabla C(\boldsymbol{\theta}_k) \to \mathbf{0}$, we get by uniqueness of limits that $\nabla C(\bar{\boldsymbol{\theta}}) = \mathbf{0}$, meaning that $\bar{\boldsymbol{\theta}}$ is a global minimizer of C. Since $\nabla C(\bar{\theta}) = 0$, using the same derivation as for $\boldsymbol{\theta}^*$ we get

$$\|\boldsymbol{\theta}_{k+1} - \bar{\boldsymbol{\theta}}\|_2^2 \leq \|\boldsymbol{\theta}_k - \bar{\boldsymbol{\theta}}\|_2^2.$$

Now $\left(\|\boldsymbol{\theta}_k - \bar{\boldsymbol{\theta}}\|_2^2\right)_k$ is non-increasing with lower bound 0, and is therefore a convergent sequence. However, we know that the subsequence given by $\|\boldsymbol{\theta}_j - \bar{\boldsymbol{\theta}}\|_2^2 \to 0$ by construction. However, a subsequence of a convergent sequence must share its limit. So $\boldsymbol{\theta}_k \to \bar{\boldsymbol{\theta}}$.

As $(C(\theta_k))_k$ is non-increasing with lower bound C^* and then converges. By continuity of C and as $\boldsymbol{\theta}_k \to \bar{\boldsymbol{\theta}}$, we have $C(\boldsymbol{\theta}_k) \to C^*$. ■

Corollary 1: [Convergence rate of gradient descent] Following the assumptions of Claim 1, $C(\boldsymbol{\theta}_k) - C^* = \mathcal{O}(1/k)$.

Proof: Following [15, Theorem 2.1.14], we combine two facts established in the proof of Claim 1. Denote $r_k \equiv \|\boldsymbol{\theta}_k - \boldsymbol{\theta}^*\|_2$ and recall

$$\|\boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}^*\|_2 \leq \|\boldsymbol{\theta}_k - \boldsymbol{\theta}^*\|_2,$$

which means $r_k \leq r_{k-1} \leq \cdots \leq r_0$. Second, denoting $\Delta_k \equiv C(\boldsymbol{\theta}_k) - C^*$, the descent lemma gave

$$\Delta_{k+1} \leq \Delta_k - c(\gamma)\|\nabla C(\boldsymbol{\theta}_k)\|_2^2.$$

Convexity of C gives, by Eq. (B1) gives,

$$C(\boldsymbol{\theta}^*) \geq C(\boldsymbol{\theta}_k) + \nabla C(\boldsymbol{\theta}_k)^T(\boldsymbol{\theta}^* - \boldsymbol{\theta}_k),$$

which can be rewritten as

$$C(\boldsymbol{\theta}_k) - C(\boldsymbol{\theta}^*) \;\equiv\; \Delta_k \leq \nabla C(\boldsymbol{\theta}_k)^T(\boldsymbol{\theta}_k - \boldsymbol{\theta}^*) \;\leq\; \|\nabla C(\boldsymbol{\theta}_k)\|_2 \, \|\boldsymbol{\theta}_k - \boldsymbol{\theta}^*\|_2 \;\leq\; r_0 \, \|\nabla C(\boldsymbol{\theta}_k)\|_2 \,.$$

Combining the inequalities, $\Delta_{k+1} \leq \Delta_k - \frac{c(\gamma)}{r_0^2}\Delta_k^2$. This means that

$$\frac{1}{\Delta_{k+1}} \geq \frac{1}{\Delta_k} + \frac{c(\gamma)}{r_0^2}\frac{\Delta_k}{\Delta_{k+1}} \geq \frac{1}{\Delta_k} + \frac{c(\gamma)}{r_0^2}.$$

Summing from 0 to k on the RHS, we get

$$\frac{1}{\Delta_{k+1}} \geq \frac{1}{\Delta_0} + \frac{c(\gamma)}{r_0^2}(k+1),$$

so $C(\boldsymbol{\theta}_k) - C^* = \Delta_k = \mathcal{O}(1/k)$. ■

Corollary 2: If C is strictly convex in addition to the assumptions of Claim 1. Then $\boldsymbol{\theta}^* = \bar{\boldsymbol{\theta}}$. Meaning that the global minimizer is unique.

Proof: Assume C is strictly convex and $\boldsymbol{\theta}_1, \boldsymbol{\theta}_2$ are both global minimizers, so $C(\boldsymbol{\theta}_1) = C(\boldsymbol{\theta}_2) = C^*$. By strict convexity

$$C\left(\frac{\boldsymbol{\theta}_1}{2} + \frac{\boldsymbol{\theta}_2}{2}\right) < \frac{1}{2}C(\boldsymbol{\theta}_1) + \frac{1}{2}C(\boldsymbol{\theta}_2) = C^*,$$

which contradicts C^* being the infimum. ■

Corollary 3: For $\gamma \in (0, 2/L)$, gradient descent converges for OLS and Ridge linear regression. The respective Lipschitz constants are

$$L_{\text{OLS}} = \mu_{\max}(\boldsymbol{H}_{\text{OLS}}), \qquad L_{\text{Ridge}} = L_{\text{OLS}} + \frac{2\lambda}{n},$$

where $H_{\text{OLS}} = \nabla^2 C_{\text{OLS}}(\boldsymbol{\theta})$ is the Hessian of $C_{\text{OLS}}(\boldsymbol{\theta})$. Further, the minimizer for Ridge is unique.

Proof: We will first prove this for OLS,

$$C(\boldsymbol{\theta}) = \frac{1}{n}\|\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y}\|_2^2$$

Obviously, C is lower bounded as the norm is non-negative. Further, we can write the norm out to get

$$\|\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y}\|_2^2 = \boldsymbol{\theta}^T\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\theta} - 2\boldsymbol{y}^T\boldsymbol{X}\boldsymbol{\theta} + \boldsymbol{y}^T\boldsymbol{y}.$$

Taking the gradient, we get

$$\nabla C(\boldsymbol{\theta}) = \frac{2}{n}\left(\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{X}^T\boldsymbol{y}\right).$$

Now, recognizing the Hessian matrix $\boldsymbol{H} = \nabla^2 C(\boldsymbol{\theta}) = \frac{2}{n}\boldsymbol{X}^T\boldsymbol{X}$, we get that for any $\boldsymbol{\theta}, \boldsymbol{\theta}'$,

$$\nabla C(\boldsymbol{\theta}) - \nabla C(\boldsymbol{\theta}') = \boldsymbol{H}(\boldsymbol{\theta} - \boldsymbol{\theta}').$$

As the Hessian is a positive semi-definite matrix,

$$\|\nabla C(\boldsymbol{\theta}) - \nabla C(\boldsymbol{\theta}')\|_2 \leq \mu_{\max}\|\boldsymbol{\theta} - \boldsymbol{\theta}'\|_2,$$

where $\mu_{\max}$ is the largest eigenvalue of the Hessian $\boldsymbol{H}$. Thus, the gradient, ∇C, is Lipschitz with constant $L_{\text{OLS}} = \mu_{\max}$.

Further, as the Hessian is positive semi-definite, C is convex. As $\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\theta} = \boldsymbol{X}^T\boldsymbol{y}$ always has a solution, the infimum is also attained. So the claim holds for OLS. For Ridge regression, we have the additional term $\frac{\lambda}{n}\boldsymbol{\theta}$, with

$$C_{\text{Ridge}}(\boldsymbol{\theta}) = C_{\text{OLS}}(\boldsymbol{\theta}) + \frac{\lambda}{n}\|\boldsymbol{\theta}\|_2^2.$$

So,

$$\boldsymbol{H}_{\text{Ridge}} = \nabla^2 C(\boldsymbol{\theta}) = \boldsymbol{H}_{\text{OLS}} + + \frac{2\lambda}{n}\boldsymbol{I},$$

which gives

$$L_{\text{Ridge}} = L_{\text{OLS}} + 2\lambda/n.$$

Again, the Hessian, $\boldsymbol{H}_{\text{Ridge}}$ is positive definite for any $\lambda > 0$, and thus C_{Ridge}, is strictly convex and coercive, meaning it is bounded below with infimum attained. As it is strictly convex the minimizer is also unique. ■

Appendix C: Gradient descent results

TABLE V: Gradient descent results for OLS and Ridge regression using $\gamma = 0.8\gamma_{\max}$. The relative parameter error is measured with respect to the corresponding closed-form solution. The quantities $\gamma_{\max}$ and γ_{opt} are calculated from the eigenvalues of the Hessian. Ridge uses $\lambda = 1$ and $n = 100$, which shifts all Hessian eigenvalues by $2\lambda/n = 0.02$.

	OLS		Ridge	
Quantity	Analytical	AD	Analytical	AD
Iterations	5948	5948	1583	1583
Relative parameter error	1.4382×10^{-6}	1.4382×10^{-6}	5.1209×10^{-7}	5.1209×10^{-7}
Optimization properties				
γ	2.8420×10^{-1}		2.8319×10^{-1}	
γ_{opt}	3.5479×10^{-1}		3.5229×10^{-1}	
$\gamma_{\max}$	3.5525×10^{-1}		3.5399×10^{-1}	
$\kappa(H)$	7.7722×10^{2}		2.0738×10^{2}	
$\max_i \lvert(\nabla C_{\text{AD}})_i - (\nabla C_{\text{analytical}})_i\rvert$	8.8818×10^{-16}		8.8818×10^{-16}	

TABLE VI: Dependence of gradient descent convergence on the learning rate for OLS and Ridge regression. The learning rate is expressed relative to the theoretical stability limit $\gamma_{\max} = 2/\mu_{\max}(H)$. Runs reaching 10 000 iterations were terminated at the prescribed maximum iteration count and should therefore not be interpreted as converged solutions.

	OLS		Ridge	
$\gamma/\gamma_{\max}$	Iterations	Relative error	Iterations	Relative error
0.01	10 000	2.495×10^{-1}	10 000	6.057×10^{-2}
0.10	10 000	2.309×10^{-2}	10 000	6.977×10^{-6}
0.30	10 000	1.340×10^{-4}	3891	1.374×10^{-6}
0.50	9154	2.305×10^{-6}	2439	8.203×10^{-7}
0.70	6724	1.645×10^{-6}	1790	5.865×10^{-7}
0.90	5337	1.279×10^{-6}	1420	4.549×10^{-7}
0.99	4889	1.162×10^{-6}	1301	4.107×10^{-7}
1.01	10 000	2.289×10^{83}	10 000	3.064×10^{83}

Appendix D: Stochastic gradient decent results

TABLE VII: OLS: Relative error to the closed-form solution after 1000 epochs for different optimizers and mini-batch sizes M. The full column corresponds to full-batch gradient descent. With learning rates in Tbl. II. Results of Fig. 18

Optimizer	$M = 1$	$M = 5$	$M = 10$	$M = 20$	$M = 50$	Full
GD	diverged	3.57×10^{-2}	2.08×10^{-2}	1.42×10^{-2}	7.14×10^{-2}	1.47×10^{-1}
Momentum	diverged	3.60×10^{-1}	3.30×10^{-2}	9.45×10^{-3}	4.29×10^{-3}	3.36×10^{-2}
AdaGrad	9.06×10^{-3}	1.06×10^{-2}	1.18×10^{-2}	1.38×10^{-2}	4.21×10^{-2}	8.00×10^{-2}
RMSprop	4.96×10^{-2}	8.51×10^{-2}	1.35×10^{-1}	1.70×10^{-1}	2.06×10^{-1}	1.13×10^{-1}
Adam	3.62×10^{-2}	1.26×10^{-2}	8.43×10^{-3}	6.54×10^{-3}	3.95×10^{-3}	6.02×10^{-9}
Mean time [s]	0.87	0.18	0.09	0.05	0.02	0.01

TABLE VIII: OLS: Comparison between full-batch gradient descent and stochastic gradient descent with mini-batch size $M = 10$ after 1000 epochs. The target relative error is 10^{-3}. With learning rates in Tbl. II.

	Final relative error			Epochs to target		Time/epoch [ms]		Time to target [ms]	
Optimizer	Full	SGD	SGD+decay	Full	SGD	Full	SGD	Full	SGD
GD	1.47×10^{-1}	2.08×10^{-2}	5.34×10^{-2}	–	–	0.007	0.080	–	–
Momentum	3.36×10^{-2}	3.30×10^{-2}	4.11×10^{-3}	–	–	0.007	0.082	–	–
AdaGrad	8.00×10^{-2}	1.18×10^{-2}	8.83×10^{-2}	–	–	0.007	0.081	–	–
RMSprop	1.13×10^{-1}	1.35×10^{-1}	3.72×10^{-1}	–	–	0.007	0.081	–	–
Adam	6.02×10^{-9}	8.43×10^{-3}	4.45×10^{-2}	589	–	0.007	0.082	4.2	–

TABLE IX: Ridge: Relative error to the closed-form solution after 1000 epochs for different optimizers and mini-batch sizes M. The full column corresponds to full-batch gradient descent. With learning rates in Tbl. II.

Optimizer	$M = 1$	$M = 5$	$M = 10$	$M = 20$	$M = 50$	Full
GD	diverged	5.48×10^{-2}	2.22×10^{-2}	8.30×10^{-3}	2.82×10^{-3}	7.07×10^{-3}
Momentum	diverged	5.76×10^{-1}	3.55×10^{-2}	1.50×10^{-2}	2.16×10^{-3}	1.57×10^{-5}
AdaGrad	3.98×10^{-3}	3.70×10^{-3}	3.89×10^{-3}	3.69×10^{-3}	5.12×10^{-3}	7.38×10^{-4}
RMSprop	8.91×10^{-2}	1.30×10^{-2}	7.21×10^{-3}	1.18×10^{-2}	2.60×10^{-2}	2.02×10^{-3}
Adam	5.43×10^{-2}	1.77×10^{-2}	1.07×10^{-2}	8.69×10^{-3}	2.05×10^{-3}	4.55×10^{-16}
Mean time [s]	0.90	0.18	0.09	0.05	0.02	0.01

TABLE X: Ridge: Comparison between full-batch gradient descent and stochastic gradient descent with mini-batch size $M = 10$ after 1000 epochs. The target relative error is 10^{-3}. With learning rates in Tbl. II.

	Final relative error			Epochs to target		Time/epoch [ms]		Time to target [ms]	
Optimizer	Full	SGD	SGD+decay	Full	SGD	Full	SGD	Full	SGD
GD	7.07×10^{-3}	2.22×10^{-2}	6.70×10^{-4}	–	–	0.009	0.087	–	–
Momentum	1.57×10^{-5}	3.55×10^{-2}	3.37×10^{-3}	534	–	0.008	0.088	4.2	–
AdaGrad	7.38×10^{-4}	3.89×10^{-3}	4.47×10^{-4}	935	–	0.008	0.087	7.4	–
RMSprop	2.02×10^{-3}	7.21×10^{-3}	2.02×10^{-1}	–	–	0.008	0.087	–	–
Adam	4.55×10^{-16}	1.07×10^{-2}	1.21×10^{-3}	429	–	0.008	0.088	3.4	–

Appendix E: Use of AI/LLM Tools

1. Tools Used

- Claude (Claude Sonnet 5, accessed via `claude.ai`, [Sept–Oct 2026])
- Claude (Claude Opus 5.5, accessed via `claude.ai`, [Sept–Oct 2026])
- ChatGPT (GPT-5.6 Sol, accessed via `chat.openai.com`, [Sept–Oct 2026])
- ChatGPT (GPT-6 Astra, accessed via `chat.openai.com`, [Sept–Oct 2026])
- GitHub Copilot Auto (integrated in VS Code and model not specified as it is Auto by default for student version)

2. Text Contributions

Section	LLM contribution level	Notes
Abstract	2	Helped finding main results to put in from the body of the report into a draft of the abstract.
Introduction	1	
Theory / Methods	2	Helped with identifying some notation clash and some inconsistencies which were later verified and referred to proper sources.
Results	2	Generated tables in LaTeX-format based on our outputs.
Discussion	1	
Conclusion	1	
Appendix A	1	
Appendix B	2	Aided in formalizing the proof and correcting mistakes. The proof itself is mainly based on the cited references.
Appendix C	2	Generated tables in LaTeX-format based on our outputs.
Appendix D	2	Generated tables in LaTeX-format based on our outputs.
Appendix E	2	Generated table-templates in LaTeX-format for us to fill out.
Bibliography	3	Used to generate BibTeX entries based on given information. Every source has been checked and verified by us.

TABLE XI: LLM contribution levels for report sections. 0 = None, 1 = Language only, 2 = Editorial, 3 = Generative.

3. Code Contributions

Docstrings: The majority of function and class docstrings in this project were generated by GitHub Copilot and ChatGPT (GPT-5.6 Sol). It was prompted with the description of the function and variables but AI was used to get correct format. Each docstring was reviewed by the authors for accuracy against the actual function and corrected if mismatched. The use of AI for this will thus not impact the scores and descriptions in the table below.

Summary: All code and text generated or assisted by an LLM was tested, verified, and understood by the authors before inclusion. All text drafted or revised with LLM assistance was read, verified, and endorsed by the authors.

File / notebook	LLM level	Description
Core modules		
`analysis.py`	0	Written independently.
`config.py`	2	Parameter names and values chosen by us, but comments of the parameters were made by an LLM.
`data.py`	0	Written independently.
`gradients.py`	1	Written independently, but an LLM helped find a normalization error in the mini-batch gradients.
`main.py`	0	Written independently.
`metrics.py`	0	Written independently.
`models.py`	2	Used for debugging and adapting when moving functions over to a dataclass structure, and for a snippet adding a singular-value cutoff to the OLS solve.
`optimizers.py`	2	An LLM found bugs in learning-rate schedule and the stopping criterion.
`plotting.py`	4	The code for plotting was mainly written by AI, based on prompts on what to plot by us.
`resampling.py`	2	LLM snippets let the cross-validation accept any estimator and added Lasso to the bootstrap.
Experiments (one file per part)		
`experiments/__init__.py`	0	Written independently.
`experiments/common.py`	2	LLM used for debugging and logic. `iterations_for_target` to give proper condition of convergence.
`experiments/ols.py`	2	Written independently except for `def run_ols_fits()`, which was written by an LLM.
`experiments/ridge.py`	0	
`experiments/bootstrap.py`	2	Formatting in `run_bootstrap` was generated by an LLM.
`experiments/cross_validation.py`	2	`get_cv_split` came from an LLM suggestion noticing an error in how we split the data. This was also implemented with the help of LLM.
`experiments/gradient_descent.py`	2	Printing in `run_gd` was written with help of an LLM.
`experiments/adaptive_gd.py`	2	Printing in `run_adaptive_gd` was written with help of an LLM.
`experiments/lasso.py`	2	Printing in `run_lasso` was written with help of an LLM.
`experiments/stochastic_gd.py`	3	`run_sgd` was made by an LLM.
`experiments/model_selection.py`	2	`run_model_selection` was made by an LLM.
Verification and figures		
`tests/__init__.py`	0	Written independently.
`tests/test_cross_validation.py`	0	Written independently.
`tests/test_gradient.py`	0	Written independently.
`supplementary_figures.ipynb`	4	Was filled with our pngs and some short captions by a Claude script. The captions were edited by us.
Other files		
`README.md`	4	We wrote a skeleton, and a start and then let Claude generate the full text based on our report and the code we gave. It was reviewed and corrected by us afterwards. It was also substantially changed by us to include other things we wanted.

TABLE XII: LLM contribution levels for code files. 0 = None, 1 = Debugging, 2 = Snippet, 3 = Skeleton, 4 = Substantial.